lsqvce 6.35.1 — API Reference

least-squares adjustment with variance component estimation

Contents

1 The task definition **2**
- 1.1 `lsqvce.TaskDefinition(Name=Value)` . . . 2
- 1.2 `addCommon(names, Name=Value)` and `sys.addLocal(names, Name=Value)` . . . 3
- 1.3 `sys = addSystem(name, Name=Value)` . . . 4
- 1.4 `sys.addStream(name, Name=Value)` . . . 5
- 1.5 `fromMatrices` and `fromBatches` . . . 5
- 1.6 `lay = task.layout()` — the columns announced . . . 6
- 1.7 Errors while declaring . . . 6

2 Prepare and Build — the functions you write **6**
- 2.1 `lsqvce.DayContext` — what the functions know . . . 7
- 2.2 `out.addArc(A, l, t, Name=Value)` — the arcs of the day . . . 8
- 2.3 The arc schedule — `Arcs` . . . 10
- 2.4 Checks while reading . . . 10

3 Noise laws **11**
- 3.1 Laws . . . 11
- 3.2 What the estimator may change — on `addStream` . . . 11
- 3.3 How a law is whitened . . . 12

4 `lsqvce.Options` **12**
- 4.1 What to estimate . . . 13
- 4.2 The loop . . . 14
- 4.3 Run control . . . 15
- 4.4 The groups . . . 16

5 Priors **18**
- 5.1 Settings . . . 18
- 5.2 Rules . . . 19
- 5.3 Where the result is reported . . . 19
- 5.4 From a parameter table . . . 20

6 `lsqvce.Results` **20**
- 6.1 Properties . . . 20
- 6.2 Methods . . . 21
- 6.3 `lsqvce.solveContributions` — a bootstrap over days . . . 22

7 The compiled declaration (engine tier) **22**
- 7.1 The `data` struct . . . 22
- 7.2 `buildDay` — the function you write . . . 27
- 7.3 Covariance specifications . . . 32
- 7.4 Priors . . . 33

8 Help of the user-tier units **35**
8.1 lsqvce.TaskDefinition . . . 35
8.2 lsqvce.SystemDefinition . . . 37
8.3 lsqvce.Noise . . . 39
8.4 lsqvce.DayContext . . . 40
8.5 lsqvce.DayData . . . 41
8.6 lsqvce.fit . . . 43
8.7 lsqvce.Options . . . 46
8.8 lsqvce.Results . . . 49
8.9 lsqvce.report . . . 52
8.10 lsqvce.layout . . . 53
8.11 lsqvce.solveContributions . . . 54

9 The engine, kernel and util tiers **56**
9.1 lsqvce.engine — drivers, accumulator, loaders, sweeps, diagnostics . . . 56
9.2 lsqvce.kernel — the numerical kernels . . . 57
9.3 lsqvce.util — plumbing . . . 58

10 Error and warning identifiers **58**

1 The task definition

Everything the toolbox knows about a problem enters through one object, a `lsqvce.TaskDefinition`, handed to `lsqvce.fit(task, opts)`. It is filled with flat calls that take name–value arguments; every call checks its arguments when it is made and names the one at fault. Nothing is read before `fit` runs. `help lsqvce.TaskDefinition` and `doc lsqvce.TaskDefinition` show the same contract.

how it is made	for
`lsqvce.TaskDefinition(Days=days)` and its `add` calls	**streaming**: arc-wise data read day by day through your `Build` functions, local unknowns eliminated on the fly, estimated stochastic models
`lsqvce.TaskDefinition.fromMatrices(A, l, Weights=p)`	**matrices**: a design matrix that fits in memory, no local unknowns, diagonal weights
`lsqvce.TaskDefinition.fromBatches(@batchFcn, NumBatches=K, NumUnknowns=u)` or `X0=x0`	**batched**: a design too large for one matrix, or a nonlinear model

1.1 `lsqvce.TaskDefinition(Name=Value)`

name	type / size	default	meaning
`Days`	double 1×D, integers, each once	`[]` (then every system sets its own `Days`)	the default calendar (e.g. MJD); the loader cache is keyed on the days
`Name`	string	`""`	a label for the run, shown by `disp(task)`

method	returns	meaning
`addCommon(names, Name=Value)`	—	common parameters
`sys = addSystem(name, Name=Value)`	`lsqvce.SystemDefinition`	an observing system
`addPrior(name, Name=Value)`	—	a prior group on the common parameters (chapter *Priors*)
`lay = layout()`	struct	the columns of A and B, with names (below)
`data = declaration()`	struct	the compiled engine-tier struct (chapter *The compiled declaration*)
`disp(task)`	—	parameter templates per scope, systems, streams and priors

1.2 `addCommon(names, Name=Value)` and `sys.addLocal(names, Name=Value)`

One call adds one **template** per name, all of one scope. A template becomes one parameter per instance of its scope.

name	type / size	default	meaning
`names`	string array	required	unique within the task (commons) or within the system (locals); a local may not share a name with a common
`Scope`	`"common"`, `"day"` (commons); `"arc"`, `"day"`, `"system"` (locals)	`"common"` / `"arc"`	over which span the parameter is constant
`Valid`	double 1×2 `[start end]`, fractional MJD, or `[]`	`[]` (always)	a `"day"` or `"arc"` template exists only on days its interval overlaps (start < d+1 and end > d); `"common"` and `"system"` templates are always present
`Info`	table, one row per name	none	your own columns; `Prepare` and `Build` read each template's row as a struct, `ctx.Common.Info{k}` / `ctx.Local.Info{k}`

scope	instantiated as	typical use
`"common"`	one column of A, always	spherical-harmonic coefficients

scope	instantiated as	typical use
`"day"` (common)	one column of A per calendar day that carries at least one arc in at least one system	a daily common parameter
`"arc"`	one column of the system's B per (day, arc)	initial state, per-arc empirical parameters; needs an arc schedule
`"day"` (local)	one column per day of the system that carries ≥ 1 arc — every calendar day without a schedule	accelerometer scale / bias / drift
`"system"`	one column for the whole adjustment	instrument constants

Column layout of A. The `"common"` templates first, in `addCommon` order; then one day block per calendar day (the sorted union of every system's days), holding that day's `"day"` commons. **Column layout of B (per system).** Per day, in calendar order: the arc blocks in sorted arc-id order (each the `"arc"` templates in `addLocal` order), then the day block; after all days one trailing `"system"` block. Your code never computes these positions: `Build` gives partials by name, or takes columns from `ctx.cols`.

1.3 `sys = addSystem(name, Name=Value)`

name	type / size	default	meaning
`name`	string, unique in the task	required	the system's name in reports
`Prepare`	function handle `@(day, ctx)`	none	the work every stream of a day shares, run once per build of the day (chapter *Prepare and Build*)
`Settings`	1×1 struct	`struct()`	anything your functions need (paths, force models); `ctx.Settings`. The field name `prepared` is reserved
`Days`	double 1×D, integers, each once	the task's `Days`	this system's calendar
`Arcs`	count, id vector, or `@(day, stream)`	none	the arc schedule (chapter *Prepare and Build*); required by `"arc"` locals
`Id`	finite double scalar or single char	its position	an identifier, unique across systems; give every system an id or none

A system is complete with one `addStream`. Its methods:

method	meaning
`addLocal(names, Name=Value)`	local parameters (above)
`addStream(name, Name=Value)`	an observation stream (below)
`addPrior(name, Name=Value)`	a prior group on this system's locals (chapter *Priors*)

1.4 `sys.addStream(name, Name=Value)`

name	type / size	default	meaning
`name`	string, a MATLAB identifier, unique in the system	required	the observation type; streams are processed in the order added
`Build`	function handle `@(day, ctx)`	required	returns the stream's arcs of one day (chapter *Prepare and Build*)
`Noise`	1×1 `lsqvce.Noise`	required	the temporal law and its level (chapter *Noise laws*)
`Estimate`	`"law"`, `"scale"`, `"none"`	`"law"` (`"scale"` for a per-row `sigma`)	what `Estimate = "noise"` may change: shape and level, the level only, nothing
`ArcFactors`	logical	`Options.ArcFactors`	per-arc variance factors for this stream
`Axes`	integer ≥ 1	1	axes per epoch; rows are epoch-major
`Arcs`	as `addSystem`	the system's	this stream's own schedule; a system gives every stream a schedule, or none

1.5 `fromMatrices` and `fromBatches`

call	arguments	meaning
`fromMatrices(A, l, Weights=p)`	A m×u, l m×1, p m×1	weights are the diagonal of P (`1./sigma.^2`); absent = unit weights
`fromBatches(@batchFcn, NumBatches=K, NumUnknowns=u)`	`[A, l, p] = batchFcn(k)`, k = 1…K	one batch of design rows, observations and weights at a time
`fromBatches(@batchFcn, NumBatches=K, X0=x0)`	`[J, w, p] = batchFcn(k, x)`	the nonlinear form: the Jacobian at x and `w = l − f(x)`; damped Gauss–Newton; u = `numel(x0)`

Both take priors by `L` only (they have no parameter names) and run with `Estimate = "none"` (variance components need streams, `lsqvce:fit:modelKind`); `Robust` selects M-estimation, except on the nonlinear form (`lsqvce:fit:notYet`). Their per-observation diagnostics are in the whitened frame, `R.diag.v = sqrt(p).*v`.

1.6 `lay = task.layout()` — the columns announced

```
lay = task.layout();                          % the same as lsqvce.layout(task)
lay.nc, lay.common(j)                         % .name, .row, .day of column j of A
lay.systems(k).name, lay.systems(k).nl
lay.systems(k).local(c)                       % .name, .row, .day, .arc of column c of B
c = lay.systems(k).col(row, day, arc);        % lookup by template row, 0 = not instantiated
c = lay.colCommon(row, day);
```

This is the column map a prior operator `L` is written in, and the one `R.toTable(task)` uses. `row` is the position of a name in `addCommon` / `addLocal` order.

1.7 Errors while declaring

identifier	raised by	meaning
`lsqvce:TaskDefinition:duplicateName`	`addCommon`, `addLocal`, `addSystem`, `addStream`, `addPrior`, `declaration`	a name used twice where it must be unique (also a local named like a common)
`lsqvce:TaskDefinition:badName`	`addStream`	a stream name that is not a MATLAB identifier
`lsqvce:TaskDefinition:badValid`	`addCommon`, `addLocal`	`Valid` is not `[]` or `[start end]` with start ≤ end
`lsqvce:TaskDefinition:infoRows`	`addCommon`, `addLocal`	`Info` has not one row per name
`lsqvce:TaskDefinition:badHandle`	`addSystem`, `addStream`	`Prepare` or `Build` is not a function handle taking `(day, ctx)`
`lsqvce:TaskDefinition:missingArgument`	`addStream`, `addPrior`, `fromBatches`	`Build`, `Noise`, `Sigma` or `NumBatches` missing
`lsqvce:TaskDefinition:priorTarget`	`addPrior`	both `On` and `L`, or neither
`lsqvce:TaskDefinition:wrongKind`	`addCommon`, `addSystem`, `addPrior`, `layout`	a streaming call on an in-memory task
`lsqvce:TaskDefinition:badArcs`	`declaration`	some streams of a system have a schedule and others none
`lsqvce:TaskDefinition:incomplete`	`declaration`	no common parameter, no system, a system without a stream, or no calendar
`MATLAB:validators:*`	every call	a value of the wrong kind (a `Scope` that does not exist, a negative `Sigma`, ...)

The checks the engines make on the compiled struct (chapter *The compiled declaration*) still run when `fit` starts; they carry `lsqvce:fit:` identifiers.

2 Prepare and Build — the functions you write

Two kinds of function produce the data on demand. Each has one home:

```
sys = task.addSystem(name, Prepare=@prepare, ...);   % P   = prepare(day, ctx)
sys.addStream(name, Build=@build, Noise=noise, ...);  % out = build(day, ctx)
```

function	runs	receives	returns
`Prepare` (optional, per system)	once per build of a day, before the first stream of that day	the day and a `lsqvce.DayContext`	anything; it reaches every `Build` of the day as `ctx.Prepared`
`Build` (required, per stream)	once per build of a day, for its stream	the day and a `lsqvce.DayContext`	the `lsqvce.DayData` of `ctx.newDay()`, filled with `addArc`

Both take exactly two arguments. Every pass over the data reads each day twice (accumulation, then the residual sweep), and an iterative run multiplies that by its passes. Hence two rules:

1. **Determinism.** The same day must give byte-identical output: no random numbers, clocks or mutable globals (seed a simulator on the day and the stream).
2. **Share the day's common work through `Prepare`.** The toolbox keeps `Options.CacheDays` built days resident (default one). Work that several streams of a day need — reading a file, integrating the orbits and their partials — goes into the system's `Prepare`, which runs once for all of them each time the day is built.

`Prepare` rule	
how often	once each time the day is built: once per run with `Options.CacheDays = Inf`; with the default `CacheDays = 1` a day is built twice per pass, so it runs as often; with `Options.Parallel` every worker holds its own copy of the loader, so it runs on every worker that builds the day
memory	the result of the current day only, whatever `CacheDays`
refused	`Settings` with a field `prepared` (`lsqvce:makeLoader:preparedField`, reported by `fit` under its own prefix, the original attached as the cause)
when it fails	`lsqvce:makeLoader:prepareDay` names the day; your error is the cause (an `lsqvce:*` error passes unchanged)

2.1 `lsqvce.DayContext` — what the functions know

property	type	meaning
`Day`	double	the day being built
`System`, `Stream`	string	the names of the system and the stream (`Stream` is `""` in `Prepare`)
`Settings`	struct	the system's `Settings`
`Prepared`	any	what `Prepare` returned for this day (`[]` in `Prepare`, or without one)
`NumCommon`	double	nc, the columns of A
`NumLocal`	double	nl, the columns of the system's B (0 without locals)

property	type	meaning
`ArcIds`	double 1×A	the stream's scheduled arcs of the day; in `Prepare`, the system's; `[]` without a schedule
`Common`, `Local`	table	one row per template: `Name`, `Scope`, `OnDay` (the template exists on this day), `Info` (a cell; the template's `Info` row as a struct, `struct()` without one)

method	returns	meaning
`newDay()`	`lsqvce.DayData`	an empty day of this stream
`cols(names)`	double 1×n	the columns of named parameters on this day: of `"common"` and `"day"` commons in A, of `"day"` and `"system"` locals in B
`cols(names, Arc=id)`	double 1×n	the same for `"arc"` locals of scheduled arc `id`

One `cols` call names commons or locals, not both.

2.2 `out.addArc(A, l, t, Name=Value)` — the arcs of the day

argument	type / size	default	meaning
`A`	m × nc	required	design of the common parameters, columns in A's layout (chapter *The task definition*)
`l`	m × 1	required	observed minus computed, raw: never whitened or weighted
`t`	m × 1	required	epoch of each row in **seconds** (the whitener needs the lags)
`Local`	n × 2 cell `{names, P; ...}`	none	local partials by name, P m × numel(names); each name's scope decides its column (this arc's, this day's or the system's block)

argument	type / size	default	meaning
`B`	m × nl, may be sparse	none	the local design as one matrix, every column from `ctx.cols`. Give `Local` or `B`, never both; neither = an arc that sees no local parameter
`Sigma`	e×1, e×d or e×d×d	none	the per-epoch accuracy of the arc (below)
`Excluded`	m × 1 logical	none	rows of **weight 0**: zero in A, the local partials and l, kept for their index (residuals, diagnostics) but not observations — out of the degrees of freedom, `rejected` in the verdict. Excluded rows are deleted from the law's covariance exactly, so they do not condition their neighbours
`Arc`	a scheduled arc id	the next	places this arc in the schedule

Arc lengths may differ freely. An arc with 0 rows is a **data gap** and is skipped.

2.2.1 `Sigma` — the per-epoch accuracy

shape	meaning	the numbers are
e × 1	scalar stream with per-epoch accuracy	standard deviations
e × d	multi-axis stream, axes uncorrelated within an epoch	standard deviations per axis
e × d × d	multi-axis stream, full per-epoch covariance (d ≥ 2)	covariance matrices

A scalar stream's `Sigma` is always **standard deviations**. MATLAB stores an e × 1 × 1 array as e × 1, so a per-epoch *variance* array would be read as sigmas: pass the square root. A full `Sigma` is composed through its Cholesky factor in the given axis order; when it varies along the arc, the composed covariance therefore depends on the frame and the axis order (XYZ against NEU: about 2 % of C). A constant or diagonal `Sigma` does not.

The axis count d is `size(Sigma, 2)`, the same for the whole day, and must agree with the stream's `Axes` when one above 1 is declared; e·d must equal the arc's row count. Rows of a multi-axis stream are **epoch-major** (x_1 y_1 z_1 x_2 …) and every arc holds whole epochs. `Sigma` is relative to the law's `sigma`: declare `lsqvce.Noise.white(1)` when `Sigma` already carries the absolute accuracy. A multi-axis day without `Sigma` is whitened with D = I and reported once per run (`lsqvce:makeLoader:noCep`); a `var` law takes no `Sigma`.

2.3 The arc schedule — `Arcs`

form	meaning
omitted	the arcs of a day are those `Build` adds (ids 1…n); days may differ in their count. `"day"` locals live on every calendar day; a calendar day with no arc in any stream is refused (`lsqvce:makeLoader:emptyDay`)
`4` or `[1 2 4]`	static: arcs 1…4, or these ids, on every day
`@(day, stream) ids`	a resolver returning the ids of a day *without reading data* (ragged schedules, gaps, manoeuvres)

With a schedule, `ctx.ArcIds` lists the day's arcs; `addArc` fills them in order, `Arc=id` places an arc out of order, and an arc the day schedules but `Build` never adds is a data gap. `"arc"` locals require a schedule. A stream may carry its own `Arcs` on `addStream`; then every stream of the system needs one (from the system or its own).

2.4 Checks while reading

identifier	meaning
`lsqvce:makeLoader:buildDay`	a `Build` threw (the message names the day and the stream); your error is attached as the cause (an `lsqvce:*` error passes unchanged)
`lsqvce:DayContext:badReturn`	`Build` returned something other than its `ctx.newDay()`
`lsqvce:DayContext:unknownName`	a name in `Local` or `cols` that is not a parameter of the system (or task), or not instantiated on this day
`lsqvce:DayContext:arcRequired`	an `"arc"` local without `Arc=id`, or with an id the day does not schedule
`lsqvce:DayContext:mixedTables`	one `cols` call names commons and locals
`lsqvce:DayContext:localAndB`	one arc gives both `Local` and `B`
`lsqvce:DayContext:badLocal`	`Local` is not `{names, P}` rows with P m × numel(names), or names a parameter twice in one arc
`lsqvce:DayContext:badArc`	`Arc` without a schedule, an unscheduled id, an arc added twice, more arcs than scheduled
`lsqvce:makeLoader:dayShape`	an arc's `A`, `B`, `l`, `t` disagree in rows
`lsqvce:accum:arcBadNc` / `arcBadNl`	`A` or `B` has the wrong number of columns
`lsqvce:makeLoader:arcCount`	a declared schedule names n arcs and the day carries a different number
`lsqvce:makeLoader:emptyDay`	without a schedule, a calendar day returned no arc in any stream
`lsqvce:makeLoader:badArc`	an arc of a multi-axis day that does not hold whole epochs
`lsqvce:makeLoader:badDayCep`	`Sigma` is malformed or does not match the arc's rows
`lsqvce:makeLoader:axisConflict`	`Axes` disagrees with `Sigma` or does not divide the row count
`lsqvce:covModel:cepOnVar`	a `var` law with a `Sigma`

identifier	meaning
`lsqvce:makeLoader:noCep`	*(warning, once per run)* a multi-axis day without `Sigma`

3 Noise laws

`sys.addStream(name, Noise=n, Estimate=e, ArcFactors=f)` declares a stream's noise in two parts: the **temporal law** with its **level**, a `lsqvce.Noise` object, and what the estimator may change about it, the stream's `Estimate` and `ArcFactors`. The per-epoch accuracy is not declared here: it travels with each arc as `Sigma` (chapter *Prepare and Build*). The toolbox composes the covariance of an arc as C = D (R_t ⊗ I_d) D. Full reference: `doc lsqvce.Noise`, `doc lsqvce.engine.covModel`.

3.1 Laws

constructor	arguments	describes	whitening cost (b rows per batch, m per arc)
`lsqvce.Noise.white(sigma)`	`sigma` scalar, or a column of per-row standard deviations	white noise	O(b)
`lsqvce.Noise.arma(ar, sigma, MA=ma)`	`ar = [1 a₁ … a_p]`, `MA` (default 1), `sigma` (innovation std)	a known whitening filter	O(b·p)
`lsqvce.Noise.psd(f, S)`	`f` (n_f×1, Hz, one-sided, **from 0**), `S` (n_f×1, unit²/Hz)	a measured noise spectrum	O(m³) per arc (Toeplitz) or O(b·p) (AR filter)
`lsqvce.Noise.acf(@rho, sigma)`	`rho(dt, opt)`, `sigma`	correlation as a function of the lag	O(m³) per arc
`lsqvce.Noise.var(ar, Sigma)`	`ar` (d×d×p), `Sigma` (d×d)	vector AR: cross-axis temporal correlation (no per-arc `Sigma`)	O(b·p·d²)
`lsqvce.Noise.custom(@covFcn)`	`covFcn(t, opt)`	any dense covariance	O(m³) per arc

Every constructor takes `MaxDenseArc=m` (default 20 000 epochs), the longest arc factorised densely. `sigma` is always a **standard deviation**, never a variance.

A `psd` grid starts at f = 0. The autocovariance integrates the piecewise-linear spectrum over the grid only, so a grid that starts higher leaves out the band below its first point. On 512 bins starting at the first nonzero bin, an AR(0.99) law lost 34 % of its variance that way. A grid whose missing band, estimated as f(1)·S(1), exceeds 10^{-6} of the variance is refused (`lsqvce:psd2acv:noZeroFrequency`). The toolbox does not guess the spectrum there: prepend f = 0 with its value, for example `[0; f]` and `[S(1); S]` for a flat extension.

3.2 What the estimator may change — on `addStream`

setting	values	default	meaning under `Estimate = "noise"`
`Estimate`	`"law"`, `"scale"`, `"none"`	`"law"`; `"scale"` for a per-row `sigma`	`"law"`: spectral shape and level are estimated; `"scale"`: the declared shape is kept, only the level is estimated; `"none"`: the stream is used exactly as declared
`ArcFactors`	logical	`Options.ArcFactors`	per-arc variance factors for this stream

A per-row `sigma` column makes the law a fixed structure: it can carry a level but not a spectral shape: its default is `"scale"`, and `Estimate = "law"` on it is refused (`lsqvce:fit:lawOnFixedStructure`). A per-arc `Sigma` does not trigger this — per-epoch accuracy together with an estimated temporal law is the supported combination.

`Options.Pool` decides which streams share one estimated spectral shape; every stream keeps its own level.

3.3 How a law is whitened

- `Options.Noise.Whitener` selects how every `psd` law whitens — the estimated law and every declared one: `"toeplitz"` (default, exact), `"ar"` or `"arma"` (a fitted filter). Check an AR fit with `lsqvce.engine.checkPSD` (the relative error of the reproduced autocovariance should be below 1e-3).
- Filter laws are applied in batches of up to 20 000 rows that overlap by the filter memory (for an MA part, until the impulse response of 1/B(z) has decayed to rounding), so the result does not depend on the batching. Dense laws (`acf`, `custom`, Toeplitz `psd`) are factorised over the whole arc (exact; $O(m^3)$, m^2 memory: 3.2 GB at 20 000 epochs). An arc longer than the law's `MaxDenseArc` is refused (`lsqvce:covModel:denseArcTooLong`): give a `psd` law a filter (`Whitener = "ar"`), cut the arcs shorter, or raise `MaxDenseArc` when the memory is there.
- An `arma` law whose MA polynomial has a root on or outside the unit circle is refused (`lsqvce:covModel:maNotInvertible`). Its exact start-up is a dense block as long as the impulse response of 1/B(z), so `MaxDenseArc` bounds that too: an MA root close to the unit circle on a long arc is refused (`lsqvce:covModel:denseArcTooLong`). The same limit bounds the whitener of an arc with many robust nuggets or excluded rows ($m \cdot k$ epoch products for k corrected epochs).
- `Whitener = "arma"` reduces the AR fit of the spectrum to an ARMA (Durbin's method, `lsqvce.kernel.ar2arma`). A reduction that comes back non-stationary or with a non-invertible MA part falls back to the AR fit, with the warning `lsqvce:ar2arma:unstable` or `lsqvce:ar2arma:nonInvertibleMA`.
- Under `Estimate = "law"` the declared law contributes only its `sigma` (the starting level): the shape is estimated from a white start.

4 `lsqvce.Options`

`lsqvce.Options` is a typed value class: every knob has a type, a range and a default, checked at assignment (`MATLAB:validators:*`) or at construction (`lsqvce:Options:*`). It says *how to run*; everything about the problem, including regularisation, is declared on the task. Three equivalent spellings:

```
opts = lsqvce.Options(Estimate="noise", MaxPasses=6);   R = lsqvce.fit(task, opts);
opts.Noise.Grid = 120;                                   % change one later (validated)
R = lsqvce.fit(task, Estimate="noise", MaxPasses=6);     % name-value pairs straight on fit
```

`disp(opts)` lists every knob, marking the defaults with *.

4.1 What to estimate

knob	values	default	meaning
`Estimate`	`"none"`, `"noise"`	`"none"`	`"none"`: one generalised least-squares solve under the declared laws. `"noise"`: the joint variance-component estimator — per pool group the spectral shape of the noise, per stream its level, per arc a factor (with `ArcFactors`), and the σ of every prior group with `estimate = "scale"`.
`Pool`	`"stream"`, `"obsType"`, `"all"`	`"stream"`	which streams share one estimated spectral shape: every (system, type) stream on its own, the streams of one observation type across systems, or all streams of the run. Every stream keeps its own level. The default was `"all"` until v6.33. Measured: pooling identical spectra gained nothing in x, while pooling different ones failed badly (efficiency 37–132, levels 2–3× off). Pool streams of the same physics deliberately (`"obsType"`).

knob	values	default	meaning
`ArcFactors`	logical	`true`	one variance factor per arc on top of the stream's level; the factor weights the solve. `false`: one level per stream. Turn off when arcs are few or short.
`Robust`	`"none"`, `"huber"`, `"igg3"`, `"bisquare"`	`"none"`	M-estimation. With `Estimate = "none"`: stand-alone IRLS on the law-whitened innovations with a MAD scale per pass (budget `RobustOpts`, factors in `R.history(end).factor`). With `Estimate = "noise"`: a robust weight per epoch inside the joint estimator, on the residuals descaled by the declared accuracy (budget `MaxPasses`, factors in `R.noise.robust.factor`). Prefer `"igg3"` or `"bisquare"` for gross blunders.

4.2 The loop

knob	values	default	meaning
`MaxPasses`	integer ≥ 1	10	pass budget of the variance-component loop and of Gauss–Newton (IRLS uses `RobustOpts.MaxIter`). Exhausting it returns the last pass with `R.stopReason = "maxPasses"` and one warning.

knob	values	default	meaning
`Tolerance`	> 0	1e-2	stop rule of the variance-component loop: every arc factor's change ≤ `Tolerance`, every spectral bin's change ≤ `Tolerance`·max(1, sqrt(200/r)), r its redundancy (at the default a tenth of the bin's standard error sqrt(2/r) where that is larger); for Gauss–Newton the relative step in units of the formal error
`Probes`	integer ≥ 1	100	Hutchinson probes for the per-lag redundancies of the joint estimator
`Seed`	`[]` or double	`[]`	seed for the probes, recorded in `R.meta.seeds`

4.3 Run control

knob	values	default	meaning
`Backend`	`"qr"`, `"normal"`	`"qr"`	square-root information (Householder QR) or normal equations (Cholesky); the same numbers to rounding, QR the safer on ill-conditioned common blocks
`CacheDays`	> 0	1	built days kept resident per system; `Inf` keeps the calendar and builds each day once per run (a built type-day is m × nc doubles); a system's `prepareDay` result is the current day's only, whatever `CacheDays`
`Verbose`	logical, or 0 / 1 / 2	`false`	screen messages: 0 silent, 1 progress (2 is accepted and acts as 1)

knob	values	default	meaning
`OnPass`	`[]` or `@(ctx)`	`[]`	inspection hook called after every pass of every iterative run (below)

4.3.1 `OnPass`

The hook receives one struct: `driver`, `pass`, `x`, `Cxx`, `sigma0`, `s0sq`; the loop state (noise loop: `S`, `Sgroups` (the bin spectra), `C`, `Cgroups`, `sigma2`, `arcKeys`, `streams`, `sigma`, `white`, `whitePval`, `maxDev`, `binRatio`, `xStep`, `skips`; IRLS: `factor`, `keys`, `scale`, `dx`; Gauss–Newton: `vPv`, `dxRel`, `lambda`); `acc`, the live accumulator; `data`, the compiled declaration (`task.declaration()` when `fit` was given a `lsqvce.TaskDefinition`); `processingOptions{k}`, system k's as declared. Return nothing to continue, `false` to stop after the pass (`R.stopReason = "userStop"`). An error in the hook arrives as `lsqvce:callUser:threw` with your error as its cause.

4.4 The groups

A group is a struct; assign a whole or partial struct, or one field. A partial struct is merged onto the defaults; an unknown field is refused (`lsqvce:Options:badGroupField`).

4.4.1 `Noise` — the estimated stochastic model

field	default	meaning
`Grid`	60	spectral bins of the estimated law between 0 and Nyquist (its resolution), an integer ≥ 2; the law is exact at every lag of every arc whatever the Grid
`Whitener`	`"toeplitz"`	how every `psd` law (the estimated one and the declared ones) whitens an arc: exact Toeplitz Cholesky over the whole arc, or a fitted AR / ARMA filter (`"ar"`, `"arma"`; any case; another value is refused, `lsqvce:Options:badWhitener`)
`Smooth`	1	moving-average half-width on the estimated spectrum, in bins (0 = none). The default 1 (it was 0) was measured better on short and long arcs: x's variance 1.02–1.11 × an oracle that knows the true covariance (0: 1.10–1.23), formal errors within 6 % of the oracle's (0: 16–27 % too large), unbiased levels, half the passes
`FreezeRedundancy`	1	a bin whose redundancy is below this keeps its value instead of being updated

field	default	meaning
`SkipThreshold`	0.2	the share of skipped arcs of a stream above which a pass is refused

4.4.2 `RobustOpts` — IRLS

field	default	meaning
`Tuning`	`[]`	tuning constant of the influence function; empty = Huber 1.345, IGG3 (1.5, 2.5), bisquare 4.685
`MaxIter`	30	IRLS passes
`Tol`	1e-8	IRLS stops when every parameter moved by less than `Tol` of its own formal sigma between two passes

4.4.3 `Diagnostics` — the per-observation tests

field	default	meaning
`Alpha`	1e-3	significance level of the outlier test
`Bonferroni`	`true`	divide α by the number of tests
`Beta`	0.2	type-II error; the power of the minimal detectable bias is $1-\beta$
`RMin`	0.01	redundancy number below which an observation is untestable
`RWeak`	0.1	redundancy number below which it is weakly controlled
`Test`	`"auto"`	`"pope"` (σ_0 estimated), `"baarda"` (σ_0 known), `"mad"` (robust scale), or automatic
`RobustCheck`	`true`	run the MAD-scale check that flags a contaminated fit
`Store`	`"all"`	keep the per-observation arrays for all observations, only `"flagged"` ones, or `"none"`
`NWorst`	1000	how many worst observations `R.report` keeps

4.4.4 `Parallel`

field	default	meaning
`Mode`	`"serial"`	`"parfor"` or `"spmd"` with the Parallel Computing Toolbox; serial with one warning without it
`DistributeDays`	`false`	distribute the day loop of the accumulation over workers (bitwise-identical fold order)
`Threshold`	8	number of days below which distribution is not attempted

4.4.5 `GaussNewton` — `fromBatches(..., X0=x0)`

field	default	meaning
`TolF`	1e-10	relative change of the objective that ends the iteration
`Lambda0`	1e-6	initial Marquardt damping (rows `sqrt(λ·diag(JᵀPJ))·dx ≈ 0`); a rejected step multiplies it by 10 (from 0: 1e-3), an accepted one divides it (below 1e-12: 0)

5 Priors

A prior is a group of pseudo-observations `L·x = x₀ + v` with weight `1/σ²`, declared on the object whose parameters it constrains:

```
task.addPrior(name, Name=Value)   % on the common parameters: L in the columns of A
sys.addPrior(name, Name=Value)    % on system sys's local parameters: L in the columns of B
```

5.1 Settings

name	meaning
`name`	the group's name, unique per owner; a local group becomes the stream `prior_<name>` of its system
`On`	a regular expression on parameter names. The toolbox builds L: one row per matched name per instance (per day for a `"day"` template, per arc for an `"arc"` template, once for `"common"` / `"system"`)
`L`	the explicit operator (q × nc, or q × nl) in the columns `task.layout()` announces. Give `On` **or** `L`, never both (`lsqvce:TaskDefinition:priorTarget`); in-memory tasks take `L` only
`Value`	a-priori values (q × 1, or a scalar); default 0

name	meaning
`Sigma`	ONE standard deviation for all rows of the group (required); relative weights inside a group go into L
`From`	the pass at which the group becomes active (default 1). A single solve reports a group with `From > 1` (`lsqvce:fit:priorNotActive`) and skips it
`Estimate`	`"none"` (default): σ fixed; `"scale"`: σ is estimated in every pass of `Estimate = "noise"` (in a run that cannot estimate it — `Estimate = "none"`, or the in-memory kinds — the group keeps its declared σ, with the warning `lsqvce:fit:priorScaleHeld`)

A pattern that matches no name, or an `L` of the wrong width, is refused when `fit` starts (`lsqvce:fit:badPrior`).

5.2 Rules

- **Several rows make σ estimable.** A group with one row has redundancy ≤ 1: its σ is re-estimated every pass from that one residual (only a group with redundancy r ≤ 0 is left unchanged, flagged in `R.noise.priorHistory`). Give a `"scale"` group several rows, or declare it `"none"`.
- **Local rows live in one day.** Every row of a local group has its non-zero columns inside one day's local block or the system block; days are coupled through common parameters only.
- **A datum is a group of its own, or several.** Minimum constraints $L = B^T$ on the null space, value 0, give the minimum-constraint solution and covariance for any σ. The datum may be split over groups (translation and rotation declared separately): every group the data cannot see belongs to it, as long as it does not overlap a group the data does see. A group that only partly lies in the null space, or overlaps an information group, is information.
- **Datum rows are not observations.** Their redundancy is 0 by construction; the diagnostics keep them in Σr and vPv but do not test or count them (`D.nTests`, `D.datumRows`; quadrant `"datum"` in `R.reliability()`). A local group's row counts as a datum row when the data see less than a millionth of it ($r \le 10^{-6}$).
- **Prior rows are declarations.** `R.snoop()` never rejects a prior row, common or local, and `R.systematic()` leaves the priors' rows out of its sequences (`D.priorRows`).

5.3 Where the result is reported

	common groups	local groups
σ per pass	`R.noise.priorHistory`	`R.history.sigma`, column `satKey|prior_<name>`
final σ	`R.noise.priors` (absolute)	`R.noise.model{k}.sigma` of stream `satKey|prior_<name>`: the factor **relative** to the group's declared σ (σ_final = declared σ × this factor)

The rows of every prior enter `R.residuals()` and the redundancy count.

5.4 From a parameter table

A row of a GRAVFIRE-style parameter table carrying `{'Tikhonov', fromPass, sigma[, label]}` in its 11th column means exactly one prior group; `template/gravfire_data.m` emits them: rows sharing a label become one group with `Estimate = "scale"`, rows without a label one fixed-σ group each.

6 `lsqvce.Results`

Every run of `lsqvce.fit` returns one `lsqvce.Results`. Its full help block, with every property and method, is in the chapter *Help of the user-tier units*; this chapter is the map.

6.1 Properties

property	size	content
`x`	nc×1	the common-parameter estimate
`sigma0`	scalar	a-posteriori standard deviation of unit weight
`stopReason`	string	`"single"`, `"converged"`, `"maxPasses"`, `"userStop"` or `"diverged"` (Gauss–Newton found no downhill step)
`converged`	logical	whether the iteration converged (true for a single solve)
`sol`	struct	the solve: `.x`, `.sigma0`, `.Cxx` (covariance scaled by σ_0^2), ...
`diag`	struct	per-observation diagnostics: residuals in the whitened frame, redundancy numbers, test statistics, minimal detectable biases, the robust check
`history`	1×I struct	one record per pass of an iterative run (empty for a single solve)
`noise`	struct	the estimated stochastic model: per stream `.stream` keys and `.model{k}` (`.sigma` the level, `.C` the law's autocovariance); `.S` / `.Sgroups` the bin spectra per pool group; `.Cgroups` the autocovariance per pool group; `.priors`, `.priorHistory`; `.checks` (when the model's assumptions fail); `.robust` (influence, scale per stream, one factor per epoch, `.truncation`, `.frozen`)
`acc`	`BlockAccumulator`	the accumulator: re-solve, inspect blocks, residuals

property	size	content
`systems`	1×M struct	what ran per system: id, nl, day list, name
`meta`	struct	provenance: `.toolboxVersion`, `.matlabRelease`, `.createdUTC`, `.seeds`, `.options` (the resolved options), `.warnings`

6.2 Methods

method	returns
`R.report()`	the verdict and its evidence, levels 0 to 4 (`lsqvce.report`)
`R.summary()`	one row per stream: final σ, law, whiteness (iterative runs)
`R.convergence()`	one row per pass: the stream levels, whiteness, `maxDev`, and the stop-rule quantities `binRatio` and `levelDev`, with the solution's step `xStep` (iterative runs)
`R.coefficients(Alpha=a)`	estimates with formal errors and (1 − a) confidence bounds
`R.toTable(task)`	one table of every parameter instance (`System`, `Name`, `Scope`, `Day`, `Arc`, `Estimate`, `Std`); `System` is `""` for commons
`R.residuals(...)`	whitened (and raw) residuals per stream, re-read from the arcs
`R.contributions()`	every (system, day)'s reduced contribution under the run's final weights, for `lsqvce.solveContributions` (one more pass over the data)
`R.contribution(...)`	the share of every group in the common solution, t_g = tr(N⁻¹N_g) (Level 2: per parameter)
`R.redundancy()`	the total redundancy n − u
`R.reliability()`	per observation: r, w, the minimal detectable bias and its quadrant
`R.systematic(...)`	unmodelled signal in the residuals: Ljung–Box, runs, binned means
`R.snoop(...)`	iterative data snooping with a clean re-solve; R is not changed
`R.compare(Rred)`	the nested-model F test against a reduced fit, under one fixed weighting: fits with estimated weights (`Estimate = "noise"`, `Robust`) are refused (`lsqvce:Results:compareEstimatedWeights`)
`R.ellipse(i, j, ...)`	the confidence ellipse of a parameter pair
`R.plot(kind)`	`"residuals"`, `"contribution"`, `"psd"`, `"convergence"`: one figure per question

The readers that need an iteration (`summary`, `convergence`) raise `lsqvce:Results:noHistory` on a single solve; those that need the accumulator (`residuals`, `contribution`) raise `lsqvce:Results:noAccumulator` on the in-memory kinds.

6.3 `lsqvce.solveContributions` — a bootstrap over days

`C = R.contributions()` streams every day once more and eliminates each day's own local parameters, so that the day's reduced normal equations remain. `out = lsqvce.solveContributions(C, c)` solves any weighted combination of the days:

`N* = sum_d c_d N_d,  n* = sum_d c_d n_d,  l'l* = sum_d c_d l'l_d,  dof* = sum_d c_d dof_d`

`c` holds one multiplicity per day of `C.days`: how often a replicate drew the day (0, 1, 2, …), or any non-negative real for a Bayesian bootstrap. On the QR backend the days' square-root rows are stacked (times √c_d); the common prior is added once, and the solve is fit's own. One replicate costs $O(D \cdot nc^2)$, independent of how many observations a day held.

output	meaning
`out.x`, `out.Cxx`, `out.sigma0`, `out.s0sq`, `out.dof`, `out.vtv`	the combination's solution
`out.weights`	`.estimate`, `.robust`, `.fixed = true`: the weights every day carries
`out.locals`	with `'Locals', true`: per drawn day `.sysID`, `.day`, `.multiplicity`, `.localIdx`, `.y`, `.Cyy`

The weights are the run's final ones, fixed: a noise run's estimated variance components and a robust run's hard weights are not re-estimated per combination (a bootstrap of the estimator under fixed weights); a robust run's σ0 is divided by its truncation factor, as fit's is. Contributions exist only where the days are separable:

identifier	cause
`lsqvce:accum:localsCoupleDays`	a local parameter takes observations on two days (declare it per day or per arc)
`lsqvce:accum:regularisedLocals`	the accumulation regularised a local block
`lsqvce:accum:dayLocalsRankDeficient`	a day leaves its own locals undetermined
`lsqvce:Results:contributionsNonlinear`	a Gauss–Newton run (its rows are linearised about x)
`lsqvce:Results:contributionsInconsistent`	the days' dof do not add up to the run's
`lsqvce:solveContributions:badContributions`	`C` is not the struct `R.contributions()` returns
`lsqvce:solveContributions:badMultiplicity`	`c` does not fit `C.days`, or is negative, or draws nothing

7 The compiled declaration (engine tier)

A `lsqvce.TaskDefinition` compiles to the struct below (`task.declaration()`), and the engines read only that struct: `lsqvce.engine.declaration`, `makeLoader` and the drivers. It is documented here for the engine tier and for what `Options.OnPass` hands your hook as `ctx.data`. A user declares a task (chapter *The task definition*); the mapping of every add-call to a field of this struct is in `RELEASE_NOTES.md` (the v6 → v7 migration table). The struct's own `buildDay` is generated by the task from the streams' `Build` functions, and its `prepareDay` from the system's `Prepare`.

7.1 The `data` struct

Everything the toolbox knows about a problem enters through one plain MATLAB struct, `data`, handed to `lsqvce.fit(data, opts)`. `fit` tells three **kinds** apart by the fields present:

kind	fields	for
streaming	`commonParams`, `days`, `systems` (+ `priors`, `paramColumns`)	arc-wise data read day by day through a `buildDay` function, local unknowns eliminated on the fly, estimated stochastic models
matrices	`A`, `l`, `p` (+ `priors`)	a design matrix that fits in memory, no local unknowns, diagonal weights
batched	`batchFcn`, `numBatches`, `numUnknowns` or `x0` (+ `priors`)	a design too large for one matrix, or a nonlinear model

`data` is a **scalar struct**; field names are case-sensitive (lowerCamelCase). An unknown field is an error (`lsqvce:fit:unknownField`), so a misspelt field cannot be ignored silently.

7.1.1 The streaming kind

field	type / size	required	meaning
`commonParams`	cell R×C, C ≥ 1	yes	the common parameter **templates** (below)
`days`	double 1×D	unless every system has its own `days`	the default calendar: integer day identifiers, each once (the loader cache is keyed on them: a non-integer list is refused, `lsqvce:fit:badDays`, a repeated day too, `lsqvce:fit:duplicateDay`)
`systems`	struct 1×M, M ≥ 1	yes	one entry per observing system (below)
`priors`	struct array	no	prior groups on the common parameters (section Priors)
`paramColumns`	struct	no	which table columns hold a row's validity and its name (below)

7.1.1.1 `commonParams` — the common parameter table One row per **template**. **Column 1 is the scope token**, columns 2…C are your own convention (degree, order, name, epoch …) and are passed through to `buildDay`:

token	instantiated as	typical use
`'common'`	one column of A, always	spherical-harmonic coefficients
`'day'`	one column per calendar day that carries at least one arc in at least one system	a daily common parameter

`'arc'` is not allowed in this table (`lsqvce:fit:badScope`): arcs belong to a system. nc, the number of columns of A, is **derived** from the expansion and never declared.

Column layout of A. The `'common'` rows first, in table order; then one day block per calendar day, in calendar order (the sorted union of every system's day list). `buildDay` receives the position of the current day's block as `info.commonDayOffset` / `info.commonDayCount`. A minimal table: `[repmat({'common'}, nc, 1), num2cell((1:nc).')]`.

7.1.1.2 `systems(k)` — one entry per observing system

All entries share the same fields; fill an unused optional field with `[]`.

field	type / size	default	meaning
`buildDay`	function handle	*(required)*	your day builder (section `buildDay`)
`covariance`	struct	*(required)*	one field per observation type holding a covariance spec (section Covariance specifications). **The fieldnames are the observation types**, in processing order; `buildDay` is called once per (day, type).
`localParams`	cell R×C	`{}`	the local parameter templates (below); not declared → nl = 0
`arcs`	double, struct or function handle	*(none)*	the arc schedule (section `buildDay`); required only by `'arc'` template rows
`days`	double 1×D	`data.days`	this system's calendar: integers, each once (`lsqvce:fit:duplicateDay`)
`id`	finite double scalar or single char	the index k	identifier, unique across systems; all ids of one kind – numbers or characters (the index default is a number: give every system an id, or none; `lsqvce:fit:mixedIdTypes`)
`name`	char / string	`''`	label used in reports
`processingOptions`	struct	`struct()`	anything `buildDay` needs (paths, force models); its 5th argument

field	type / size	default	meaning
`prepareDay`	function handle	none	the work every stream of a day shares, run once per build of the day, before its first stream (not once per stream); its result reaches `buildDay` as `procOpt.prepared`, inside its 5th argument (section `buildDay`, *What all streams of a day share*)
`priors`	struct array	none	prior groups on this system's local parameters (section Priors)

`buildDay` + `covariance` is a complete system.

7.1.1.3 `localParams` — the local parameter table Column 1 is the scope token, columns 2…C your convention:

token	one block per …	typical use
`'arc'`	(day, arc) of the system	initial state, per-arc empirical parameters
`'day'`	day of the system that carries ≥ 1 arc — every calendar day when no schedule is declared	accelerometer scale / bias / drift
`'system'`	whole adjustment	instrument constants

`'arc'` rows require a declared arc schedule; `'day'` rows without a schedule live on every calendar day of the system; `'system'`-only tables need neither. nl is derived.

Column layout of B (per system). Per day, in calendar order: the arc blocks in sorted arc-id order (each block = the `'arc'` rows in table order), then the `'day'` block; after all days one trailing `'system'` block. A day whose declared schedule names no arc gets no block at all. `buildDay` never computes an offset: every offset arrives in its `info` argument.

A template has the same rows for every arc. Arcs with genuinely different parameter counts are an engine-tier feature (`lsqvce.engine.makeLoader` with `LocalsPerArc`).

7.1.1.4 `paramColumns` — validity and names

field	meaning
`start`, `end`	the columns of both tables that hold a row's validity (`[MJD seconds]` pairs or fractional MJD; `[]` = always valid; any other entry, or a start after its end, is refused: `lsqvce:fit:badValidity`). A `'day'` / `'arc'` row is then instantiated only on calendar days its validity overlaps (start < d+1 and end > d); `'common'` / `'system'` rows are always present.
`name`	the column holding a row's name, matched by the `group` field of a prior (default 2)

Without `paramColumns` every row is valid always. `buildDay`'s `info` then lists which template rows exist on the day (`info.arcRows`, `info.dayRows`, `info.sysRows`, `info.commonDayRows`).

7.1.1.5 `lsqvce.layout(data)` — the columns announced

```
lay = lsqvce.layout(data);
lay.nc, lay.common(j)                          % template row (and day) of column j of A
lay.systems(k).nl, lay.systems(k).local(c)     % (row, day, arc) of column c of B
c = lay.systems(k).col(row, day, arc);          % lookup, 0 = not instantiated
c = lay.colCommon(row, day);
```

This is the column map a prior operator `L` is written in, and the one `R.toTable` uses to write estimates back into your tables.

7.1.2 The matrices kind

field	size	meaning
`A`	m×u	design matrix
`l`	m×1	observed minus computed
`p`	m×1	weights (the diagonal of P); absent = unit weights
`priors`	struct array	prior groups (section Priors)

7.1.3 The batched kind

field	size	meaning
`batchFcn`	function handle	`[A, l, p] = batchFcn(k)`, k = 1…`numBatches`: one batch of design rows, observations and weights. With `x0`: `[A, w, p] = batchFcn(k, x)` returns the Jacobian at `x` and `w = l − f(x)`.
`numBatches`	scalar	the number of batches
`numUnknowns`	scalar	u, linear form only
`x0`	u×1	the starting point; makes the problem **nonlinear** (damped Gauss–Newton). u = `numel(x0)`; `numUnknowns` must then be absent (`lsqvce:fit:uCount`).

field	size	meaning
`priors`	struct array	prior groups (section Priors)

Both in-memory kinds run with `Estimate = "none"` (variance components need the streams of a streaming system, `lsqvce:fit:modelKind`); `Robust` selects M-estimation, except on the nonlinear form: `Robust` with `x0` is refused (`lsqvce:fit:notYet`; linearise first). Their per-observation diagnostics are in the whitened frame, `R.diag.v = sqrt(p).*v`.

7.2 `buildDay` — the function you write

7.2.1 Signature

`fit` passes as many of these arguments as the handle declares (`nargin`); a `varargin` builder receives all six:

```
D = buildDay(day, obsType)
D = buildDay(day, obsType, common, local, procOpt)
D = buildDay(day, obsType, common, local, procOpt, info)
```

argument	what it is
`day`	one entry of the system's calendar (numeric)
`obsType`	one entry of `fieldnames(covariance)` (char)
`common`	`data.commonParams`
`local`	this system's `localParams` (or `{}`)
`procOpt`	this system's `processingOptions`; with a `prepareDay`, also the field `prepared` (its result — the name is then reserved)
`info`	this day's expansion: every column offset of A and B (below); `[]` when nothing is scoped

`fit` calls `buildDay` once per (day, observation type). Every pass over the data reads each day twice (accumulation, then the residual sweep), and an iterative run multiplies that by its passes. Hence two rules:

1. **Determinism.** The same arguments must give byte-identical output: no random numbers, clocks or mutable globals.
2. **Share the day's common work through `prepareDay`.** The toolbox keeps `Options.CacheDays` built days resident (default one). Work that several observation types of a day need — reading a file, integrating the orbits and their partials — goes into the system's `prepareDay`, which runs once for all of them each time the day is built (next section).

7.2.2 Where the handle goes — one per system

`buildDay` is not an option of `fit`; it is part of the data. Every entry of `data.systems` carries its own handle in the field `buildDay`, next to its own `covariance`, `localParams`, `days`, `arcs` and `processingOptions`, and `fit` calls each system's handle on that system's calendar only. `lsqvce.Options` says how one run proceeds (`OnPass` among it); the `data` struct says what is read and by whom.

your situation	what to do
several instruments or satellites in **one** adjustment	one entry of `data.systems` each, each with its own handle; they share only the common parameters

your situation	what to do
one reader, several setups (data directory, background models, editing thresholds)	pass the setup in as data: plain values in `processingOptions` (the 5th argument), or a closure that binds a configuration object
data sets that do **not** share common parameters	one `data` struct each, one `fit` call each

```
% two instruments, two readers, one adjustment
data.systems(1).buildDay = @buildDayKbr;
data.systems(1).covariance.kbr = struct('type', 'diag', 'sigma', 1e-7);
data.systems(2).buildDay = @buildDayPos;
data.systems(2).covariance.pos = struct('type', 'diag', 'sigma', 0.02);

% one reader, the setup as data: processingOptions is the 5th argument ...
s.buildDay          = @myBuildDay;     % D = myBuildDay(day, obsType, common, local, procOpt)
s.processingOptions = struct('dataDir', '/data/setupA', 'gravityModel', 'GOCO06s');

% ... or bound by a closure (the examples do this)
cfg = mySetup('setupB');
s.buildDay = @(day, obsType, common, local, procOpt, info) ...
                 myBuildDay(cfg, day, obsType, common, local, procOpt, info);
```

A closure declares its own argument count, and that count is what `fit` passes (`nargin`, above): declare `info` when your rows need the column offsets of local or `'day'` parameters. What the handle captures must not change during the run — it is called again on every pass (rule 1). It is also the place for a cache of your own that outlives a pass — the raw files of a day, say, when they are smaller than the built rows; what the streams of one day share goes into `prepareDay` (rule 2). `buildDay` that is not a function handle is refused before any data is read (`lsqvce:fit:builderKind`).

7.2.3 What all streams of a day share — `prepareDay`

`buildDay` is called once per (day, observation type). When the streams of a day rest on one expensive computation — the reference orbits and the variational partials of both satellites, used by the kinematic orbits **and** the range-rates — give the system a `prepareDay` handle. `fit` calls it **once per build of a day**, before the first stream of that day is built, and hands its result to every `buildDay` of that day as `procOpt.prepared`:

```
s.prepareDay = @prepareGrace;       % P = prepareGrace(day, common, local, procOpt, info)
s.buildDay   = @buildDayGrace;      % D = buildDayGrace(day, obsType, common, local, procOpt, int

function P = prepareGrace(day, ~, ~, procOpt, info)
    P.C = integrateWithPartials('C', day, procOpt, info);   % r, v and d[r;v]/dp, whole day
    P.D = integrateWithPartials('D', day, procOpt, info);
end

function D = buildDayGrace(day, obsType, ~, ~, procOpt, info)
    P = procOpt.prepared;               % what prepareGrace returned for this day
    switch obsType
        case 'posC', D = kinematicRows(P.C, day, info);
        case 'posD', D = kinematicRows(P.D, day, info);
        case 'kbr',  D = rangeRateRows(P.C, P.D, day, info);
    end
end
```

rule	
arguments	as many of `(day, common, local, procOpt, info)` as the handle declares, none to all five; `info` is the same day slice `buildDay` gets, `procOpt` the plain `processingOptions`
how often	once each time the day is built, for all its streams: once per run with `Options.CacheDays = Inf`; with the default `CacheDays = 1` a day is built twice per pass (accumulation, residual sweep), so the hook runs as often; with `Options.Parallel` every worker holds its own copy of the loader, so it runs on every worker that builds the day
memory	the result of the current day only, whatever `CacheDays`: the toolbox keeps the built streams, not the result
determinism	like `buildDay`: it runs again whenever its day is built again
refused	a handle of another class (`lsqvce:fit:builderKind`); a `buildDay` that declares fewer than five arguments and so never sees `procOpt` (`lsqvce:makeLoader:prepareUnused`); a `processingOptions` that already has a field `prepared` (`lsqvce:makeLoader:preparedField`) — `fit` reports both under its own prefix, the original attached as the cause
when it fails	`lsqvce:makeLoader:prepareDay` names the day; your error is the cause (an `lsqvce:*` error passes unchanged); a handle that does not resolve is reported the same way

7.2.4 The `info` argument

```
info.arcId            % 1xA  ids of this day's arcs
info.offsetOf         % @(arcId) -> column offset of that arc's block in B
info.count            % 1xA  columns of each arc block (= number of 'arc' rows)
info.dayOffset        % 1x1  this day's 'day' block offset in B ([] if none)
info.dayCount         % 1x1  its width (0 if none)
info.sysOffset        % 1x1  the 'system' block offset in B
info.sysCount         % 1x1  its width
info.nl               % 1x1  total local count of the system
info.nc               % 1x1  total common count
info.commonCount      % 1x1  width of A's leading 'common' block
info.commonDayOffset  % 1x1  this day's common day-block offset in A ([] if none)
info.commonDayCount   % 1x1  its width (0 if none)
info.arcRows, info.dayRows, info.sysRows, info.commonDayRows
                      %      template rows that exist on this day (with paramColumns)
```

Fill `B(rows, info.offsetOf(id) + (1:k))` and `A(:, info.commonDayOffset + (1:w))`.

7.2.5 The return value: one cell per arc

Every field is a **1 × nArc cell**; cell a is arc a. A day with one arc returns `{A}`; a day without data returns empty cells.

field	each cell	meaning
`A`	m_a × nc	design matrix of the common unknowns
`B`	m_a × nl	design matrix of the local unknowns; may be `sparse`
`l`	m_a × 1	observed minus computed (the reduced observation)
`t`	m_a × 1	epoch of each row in **seconds** (the whitener needs the lags)
`Cep`	n_a×1, n_a×d or n_a×d×d	*optional* — the per-epoch accuracy of the arc (below)
`nAxis`	scalar	*optional* — the axis count of a multi-axis stream without `Cep`
`excluded`	m_a × 1 logical	*optional* — rows of **weight 0**: zero in `A`, `B` and `l`, kept for their index (residuals, diagnostics) but not observations — out of the degrees of freedom, `rejected` in the verdict. Excluded rows are deleted from the law's covariance exactly, so they do not condition their neighbours: whole epochs, single axes of an epoch, and every row of a `var` law. Only independent rows (a `diag` law without a full `Cep`) need nothing. Keep a rejected epoch on the grid this way rather than removing it

Arc lengths may differ freely. An arc whose cells have 0 rows is a **data gap** and is skipped. Return **raw, unwhitened** matrices: the toolbox whitens every arc from the covariance spec and reweights it during the variance-component loop. No other field is allowed (`lsqvce:makeLoader:badDay`).

7.2.5.1 `Cep` — the per-epoch accuracy

shape	meaning	the numbers are
n_a × 1	scalar stream with per-epoch accuracy	standard deviations
n_a × d	multi-axis stream, axes uncorrelated within an epoch	standard deviations per axis
n_a × d × d	multi-axis stream, full per-epoch covariance (d ≥ 2)	covariance matrices

A scalar stream's `Cep` is always **standard deviations**. MATLAB stores an n_a × 1 × 1 array as n_a × 1, so a per-epoch *variance* array would be read as sigmas: pass the square root. A full `Cep` is composed through its Cholesky factor in the given axis order. When it varies along the arc, the composed covariance therefore depends on the frame and the axis order (XYZ against NEU: about 2 % of C). A constant or diagonal `Cep` does not.

The axis count d is `size(Cep, 2)`, the same for the whole day; n_a·d must equal the arc's row count. Rows of a multi-axis stream are **epoch-major** (x_1 y_1 z_1 x_2 …) and every arc holds whole epochs. `Cep` is relative to the level `sigma` of the covariance spec: declare `sigma = 1` when `Cep` already carries the absolute accuracy. A multi-axis day without `Cep` is whitened with D = I and reported once per run (`lsqvce:makeLoader:noCep`); a `var` law takes no `Cep`.

7.2.6 The arc schedule — `systems(k).arcs`

form	meaning
omitted	the arcs of a day are the cells `buildDay` returns (ids 1…nArc); days may differ in their count. `'day'` rows live on every calendar day; a calendar day returning no arc in any stream is refused (`lsqvce:makeLoader:emptyDay`).
`4` or `[1 2 4]`	static: these arc ids on every day
`struct('kbr', 2, 'pos', 4)`	static, per observation type
`@(day, obsType) ids`	a resolver returning the ids of a day *without reading data* (ragged schedules, gaps, manoeuvres)

With a declared schedule cell a is the a-th scheduled id, and the number of cells must match (`lsqvce:makeLoader:arcCount`). An arc a scheduled day does not carry is returned as an empty cell. `'arc'` template rows require a declared schedule.

7.2.7 Checks while reading

identifier	meaning
`lsqvce:makeLoader:buildDay`	`buildDay` threw; its error is attached as the cause (an `lsqvce:*` error passes unchanged)
`lsqvce:makeLoader:badDay`	a required field is missing or not a 1×nArc cell, or an unknown field is present, or `excluded` marks a row that is not zero
`lsqvce:makeLoader:dayShape`	the cells disagree in number, or an arc's `A`, `B`, `l`, `t` disagree in rows
`lsqvce:accum:arcBadNc` / `arcBadNl`	`A` or `B` has the wrong number of columns
`lsqvce:makeLoader:arcCount`	a declared schedule names n arcs and the day returned a different number of cells
`lsqvce:makeLoader:emptyDay`	without a schedule, a calendar day returned no arc in any stream
`lsqvce:makeLoader:badArc`	an arc id outside the declared schedule, or an arc of a multi-axis day that does not hold whole epochs
`lsqvce:makeLoader:badDayCep`	`Cep` is malformed or does not match the arc's rows
`lsqvce:makeLoader:axisConflict`	`nAxis` disagrees with `Cep` or does not divide the row count
`lsqvce:makeLoader:cepTwice`	the covariance spec carries a `Cep` and the day returns one
`lsqvce:covModel:cepOnVar`	a `var` law with a `Cep`
`lsqvce:makeLoader:noCep`	*(warning, once per run)* a multi-axis day without `Cep`

Errors in the declaration itself are raised by `fit` before any data is read and carry `lsqvce:fit:` identifiers (`unknownField`, `noSystems`, `noCommon`, `missingField`, `noDays`, `badDays`, `duplicateDay`, `badId`, `mixedIdTypes`, `duplicateSystem`, `builderKind`, `badArcs`, `badScope`, …).

7.3 Covariance specifications

`data.systems(k).covariance.(obsType)` is one struct per observation type. Its `type` selects the **temporal law** — how the errors of one stream correlate down the epochs — and `sigma` its **level**. The per-epoch accuracy is not declared here: it travels with the day as `Cep` (section `buildDay`). The toolbox composes the covariance of an arc as C = D (R_t ⊗ I_d) D. Full reference: `doc lsqvce.engine.covModel`.

7.3.1 Laws

`type`	fields	describes	whitening cost (b rows per batch, m per arc)
`'diag'`	`sigma` (scalar, or a vector of per-row standard deviations)	white noise	O(b)
`'arma'`	`ar` = `[1 a₁ … a_p]`, `ma` (default 1), `sigma` (innovation std)	a known whitening filter	O(b·p)
`'psd'`	`f` (n_f×1, Hz, one-sided, **from 0**), `S` (n_f×1, unit²/Hz)	a measured noise spectrum	O(m³) per arc (Toeplitz) or O(b·p) (AR filter)
`'acf'`	`acf = @(dt, opt)`, `sigma`	correlation as a function of the lag	O(m³) per arc
`'var'`	`ar` (d×d×p), `sigma` (d×d)	vector AR: cross-axis temporal correlation (no `Cep`)	O(b·p·d²)
`'custom'`	`fcn = @(t, opt)`	any dense covariance	O(m³) per arc

`sigma` is always a **standard deviation**, never a variance.

A `psd` grid starts at f = 0. The autocovariance integrates the piecewise-linear spectrum over the grid only, so a grid that starts higher leaves out the band below its first point. On 512 bins starting at the first nonzero bin, an AR(0.99) law lost 34 % of its variance that way. A grid whose missing band, estimated as f(1)·S(1), exceeds 10^{-6} of the variance is refused (`lsqvce:psd2acv:noZeroFrequency`). The toolbox does not guess the spectrum there: prepend f = 0 with its value, for example `[0; f]` and `[S(1); S]` for a flat extension.

7.3.2 Policy fields

field	values	default	meaning under `Estimate = "noise"`
`estimate`	`"law"`, `"scale"`, `"none"`	`"law"`	`"law"`: spectral shape and level are estimated; `"scale"`: the declared shape is kept, only the level is estimated; `"none"`: the stream is used exactly as declared
`arcFactors`	logical	`Options.ArcFactors`	per-arc variance factors for this stream

A per-row `sigma` vector makes the spec a fixed structure: it can carry a level but not a spectral shape, so `estimate = "law"` on it is refused (`lsqvce:fit:lawOnFixedStructure`). A `Cep` does not trigger this — per-epoch accuracy together with an estimated temporal law is the supported combination.

`Options.Pool` decides which streams share one estimated spectral shape; every stream keeps its own level.

7.3.3 How a law is whitened

- `Options.Noise.Whitener` selects how every `psd` law whitens — the estimated law and every declared one: `"toeplitz"` (default, exact), `"ar"` or `"arma"` (a fitted filter). Check an AR fit with `lsqvce.engine.checkPSD` (the relative error of the reproduced autocovariance should be below 1e-3).
- Filter laws are applied in batches of up to 20 000 rows that overlap by the filter memory (for an MA part, until the impulse response of 1/B(z) has decayed to rounding), so the result does not depend on the batching. Dense laws (`acf`, `custom`, Toeplitz `psd`) are factorised over the whole arc (exact; $O(m^3)$, m^2 memory: 3.2 GB at 20 000 epochs). An arc longer than the spec's `maxDenseArc` (default 20 000 epochs) is refused (`lsqvce:covModel:denseArcTooLong`): give a `psd` law a filter (`Whitener = "ar"`), cut the arcs shorter, or raise `maxDenseArc` when the memory is there.
- An `arma` law whose MA polynomial has a root on or outside the unit circle is refused (`lsqvce:covModel:maNotInvertible`). Its exact start-up is a dense block as long as the impulse response of 1/B(z), so `maxDenseArc` bounds that too: an MA root close to the unit circle on a long arc is refused (`lsqvce:covModel:denseArcTooLong`). The same limit bounds the whitener of an arc with many robust nuggets or deleted rows (m·k epoch products for k corrected epochs).
- `Whitener = "arma"` reduces the AR fit of the spectrum to an ARMA (Durbin's method, `lsqvce.kernel.ar2arma`). A reduction that comes back non-stationary or with a non-invertible MA part falls back to the AR fit, with the warning `lsqvce:ar2arma:unstable` or `lsqvce:ar2arma:nonInvertibleMA`.
- Under `estimate = "law"` the declared law contributes only its `sigma` (the starting level): the shape is estimated from a white start.

7.4 Priors

A prior is a group of pseudo-observations $L \cdot x = x_0 + v$ with weight $1/\sigma^2$, declared in your columns:

```
data.priors(j)            = struct(...)   % on the common parameters: L in the columns of A
data.systems(k).priors(l) = struct(...)   % on system k's local parameters: L in the columns of E
```

7.4.1 Fields

field	meaning
`group`	a regular expression on the name column of the table (`data.paramColumns.name`, default column 2). The toolbox builds L: one row per matched template row per instance (per day for a `'day'` row, per arc for an `'arc'` row, once for `'common'` / `'system'`).
`L`	the explicit operator (q × nc, or q × nl) in the columns `lsqvce.layout(data)` announces. Give `group` **or** `L`, never both.
`x0`	a-priori values (q × 1, or a scalar); default 0
`sigma`	ONE standard deviation for all rows of the group; relative weights inside a group go into L
`from`	the pass at which the group becomes active (default 1). A single solve reports a group with `from > 1` (`lsqvce:fit:priorNotActive`) and skips it.
`name`	label (default: the pattern); a local group becomes the stream `prior_<name>` of its system
`estimate`	`"none"` (default): σ fixed; `"scale"`: σ is estimated in every pass of `Estimate = "noise"` (in a run that cannot estimate it — `Estimate = "none"`, or the in-memory kinds — the group keeps its declared σ, with the warning `lsqvce:fit:priorScaleHeld`)

Malformed fields, or an `L` of the wrong width, are refused by name (`lsqvce:fit:badPrior`).

7.4.2 Rules

- **Several rows make σ estimable.** A group with one row has redundancy ≤ 1: its σ is re-estimated every pass from that one residual (only a group with redundancy r ≤ 0 is left unchanged, flagged in `R.noise.priorHistory`). Give a `"scale"` group several rows, or declare it `"none"`.
- **Local rows live in one day.** Every row of a local group has its non-zero columns inside one day's local block or the system block; days are coupled through common parameters only.
- **A datum is a group of its own, or several.** Minimum constraints $L = B^T$ on the null space, `x0 = 0`, give the minimum-constraint solution and covariance for any σ. The datum may be split over groups (translation and rotation declared separately): every group the data cannot see belongs to it, as long as it does not overlap a group the data does see. A group that only partly lies in the null space, or overlaps an information group, is information.
- **Datum rows are not observations.** Their redundancy is 0 by construction; the diagnostics keep them in Σr and vPv but do not test or count them (`D.nTests`, `D.datumRows`; quadrant `"datum"` in `R.reliability()`). A local group's row counts as a datum row when the data see less than a millionth of it ($r \le 10^{-6}$).
- **Prior rows are declarations.** `R.snoop()` never rejects a prior row, common or local, and `R.systematic()` leaves the priors' rows out of its sequences (`D.priorRows`).

7.4.3 Where the result is reported

	common groups	local groups
σ per pass	`R.noise.priorHistory`	`R.history.sigma`, column `satKey|prior_<name>`
final σ	`R.noise.priors` (absolute)	`R.noise.model{k}.sigma` of stream `satKey|prior_<name>`: the factor **relative** to the group's declared σ (σ_final = declared σ × this factor)

The rows of every prior enter `R.residuals()` and the redundancy count.

7.4.4 From a parameter table

A row of a GRAVFIRE-style parameter table carrying `{'Tikhonov', fromPass, sigma[, label]}` in its 11th column means exactly one such struct; `template/gravfire_data.m` emits them: rows sharing a label become one `group` with `estimate = "scale"`, rows without a label one fixed-σ group each.

8 Help of the user-tier units

The help blocks of the eleven user-tier units, verbatim: `help lsqvce.<name>` shows the same text and `doc lsqvce.<name>` the same page. Generated from the code at build time.

8.1 lsqvce.TaskDefinition

LSQVCE.TASKDEFINITION Declare a least-squares task: parameters, systems, streams, priors.

```
LSQVCE.TASKDEFINITION  Declare a least-squares task: parameters, systems, streams, priors.

  task = lsqvce.TaskDefinition(Days=days)
  task = lsqvce.TaskDefinition(Days=days, Name=label)
  task = lsqvce.TaskDefinition.fromMatrices(A, l)
  task = lsqvce.TaskDefinition.fromMatrices(A, l, Weights=p)
  task = lsqvce.TaskDefinition.fromBatches(@batchFcn, NumBatches=K, NumUnknowns=u)
  task = lsqvce.TaskDefinition.fromBatches(@batchFcn, NumBatches=K, X0=x0)

  A TaskDefinition is a handle object you fill with flat add-calls. Every
  call checks its arguments at once and names the one at fault; nothing is
  read until LSQVCE.FIT(task, ...) runs. disp(task) prints what was
  declared: parameter templates per scope, systems, streams and priors.

  WHERE THE DAILY WORK GOES. A system's Prepare function (addSystem) runs
  once per system and day, before its streams; each stream's Build function
  (addStream) runs once per stream and day and returns that day's arcs.
  Both are called as  f(day, ctx)  with a LSQVCE.DAYCONTEXT.

  The task compiles to the declaration struct of the engines
  (task.declaration()); the estimate, the noise model and every diagnostic
  are the engines' own -- this class does no arithmetic.

  METHODS
    addCommon(task, names, Name=Value)    common parameters (every system sees them)
      Scope  "common" (default) | "day"   constant over the run, or one per calendar day
```

```
   Valid  [start end] fractional MJD | [] (default: always)   when a "day" row exists
    Info   table, one row per name      your own columns, readable in ctx.Common
sys = addSystem(task, name, Name=Value)  an observing system (LSQVCE.SYSTEMDEFINITION)
   Prepare  @(day, ctx) -> anything    once per system and day; result = ctx.Prepared
    Settings struct                     paths, models, thresholds; ctx.Settings
    Days     integer row vector         the system's calendar (default: the task's)
    Arcs     count | id vector | @(day, stream) -> ids   the arc schedule; needed by
                                        Scope="arc" locals (default: none, the arcs
                                        are what Build returns)
   Id      number | single character  an identifier (give every system one, or none)
  addPrior(task, name, Name=Value)      a prior group on the common parameters
    On       regular expression on parameter names (give On or L)
    L        q x nc operator in the columns task.layout() announces
    Value    a-priori value, scalar or q x 1 (default 0)
    Sigma    one standard deviation for the group (required)
    Estimate "none" (default) | "scale"   estimate the group's sigma
    From     pass at which the group becomes active (default 1)
  lay = layout(task)                    the columns of A and B, with names
  data = declaration(task)              the engine-tier declaration struct
STATIC CONSTRUCTORS (in-memory tasks; both take addPrior with L only)
  fromMatrices(A, l, Weights=p)        A m x u, l m x 1, p m x 1 weights (1./sigma.^2)
  fromBatches(@batchFcn, NumBatches=K, NumUnknowns=u)   [A, l, p] = batchFcn(k)
  fromBatches(@batchFcn, NumBatches=K, X0=x0)          [J, w, p] = batchFcn(k, x)

INPUT
  Days  1 x D double   integer day identifiers (e.g. MJD), each once
  Name  1x1 string     a label for the run (default "")
OUTPUT
  task  1x1 lsqvce.TaskDefinition   the declaration, empty until filled

ACCURACY
  Not applicable: bookkeeping -- names, scopes and handles turned into the
  engines' declaration; no floating-point result. A fit of a
  TaskDefinition equals the fit of the hand-written declaration BITWISE
  (test_taskDefinition, testCompiledTaskFitsBitwiseLikeTheStruct).

ERRORS  (raised by the call that makes the mistake, before any data is read)
  lsqvce:TaskDefinition:duplicateName    a name is used twice: commons, the
                                         locals of a system, a local named like
                                         a common, systems, streams, priors
  lsqvce:TaskDefinition:badName          a stream name that is not a valid
                                         MATLAB identifier (it becomes a field)
  lsqvce:TaskDefinition:badValid         Valid is not [] or [start end] with
                                         start <= end
  lsqvce:TaskDefinition:infoRows         Info has not one row per name
  lsqvce:TaskDefinition:badHandle        Prepare or Build is not a function
                                         handle taking (day, ctx)
  lsqvce:TaskDefinition:missingArgument  a required Name-Value argument is
                                         missing (Build, Noise, Sigma, NumBatches)
  lsqvce:TaskDefinition:priorTarget      a prior gives both On and L, or neither
  lsqvce:TaskDefinition:wrongKind        addCommon / addSystem / a prior with On
                                         on an in-memory task
  lsqvce:TaskDefinition:badArcs          a system mixes scheduled and
                                         unscheduled streams
  lsqvce:TaskDefinition:incomplete       declaration() of a task without a
```

```
                                        common parameter, a system, a stream
                                        per system, or a calendar
  MATLAB:validators:*                   a Name-Value value of the wrong kind
                                        (Scope, Estimate, Days, ...)

EXAMPLE
  task = lsqvce.TaskDefinition(Days=1:3);
  task.addCommon(["a" "b"]);
  sys = task.addSystem("line");
  sys.addStream("y", Build=@buildLine, Noise=lsqvce.Noise.white(0.1));
  R = lsqvce.fit(task);
  % with  function out = buildLine(day, ctx)
  %           t = (0:99).' + 100 * day;   out = ctx.newDay();
  %         out.addArc([ones(100, 1), t / 1e3], 2 + 3e-3 * t + 0.1 * randn(100, 1), t);
  %       end

See also LSQVCE.SYSTEMDEFINITION, LSQVCE.NOISE, LSQVCE.DAYCONTEXT, LSQVCE.DAYDATA,
         LSQVCE.FIT, LSQVCE.LAYOUT, LSQVCE.OPTIONS
```

8.2 lsqvce.SystemDefinition

LSQVCE.SYSTEMDEFINITION One observing system of a task: local parameters, streams, priors.

```
LSQVCE.SYSTEMDEFINITION  One observing system of a task: local parameters, streams, priors.

  sys = task.addSystem(name, Name=Value)        made by LSQVCE.TASKDEFINITION only
  sys.addLocal(names, Name=Value)
  sys.addStream(name, Build=@build, Noise=noise, Name=Value)
  sys.addPrior(name, Name=Value)

  A system is one satellite, one receiver, or a pair whose observation
  depends on the local parameters of both (a range-rate between two
  spacecraft is ONE system). Systems share only the common parameters;
  each has its own locals, streams, calendar and priors.

  WHERE THE DAILY WORK GOES. The system's Prepare (given to addSystem) runs
  once per day, before the first stream of the day; each stream's Build
  (given to addStream) runs once per day and returns the stream's arcs:
      P   = prepare(day, ctx)       % its result reaches every Build as ctx.Prepared
      out = build(day, ctx)         % out = ctx.newDay(); out.addArc(...)
  (LSQVCE.DAYCONTEXT, LSQVCE.DAYDATA). With Options.CacheDays = 1 a day is
  built twice per pass (accumulation, residual sweep); CacheDays = Inf builds
  it once per run. Both functions must return the same result for the same day.

  METHODS
    addLocal(sys, names, Name=Value)      local parameters, eliminated arc by arc
      Scope  "arc" (default) | "day" | "system"   constant over one arc, one day of
                                          the system, or the whole run; "arc" needs
                                          an Arcs schedule on the system or stream
      Valid  [start end] fractional MJD | [] (default: always)   when the row exists
      Info   table, one row per name      your own columns, readable in ctx.Local
    addStream(sys, name, Name=Value)      an observation stream (one observation type)
      Build      @(day, ctx) -> LSQVCE.DAYDATA   required; once per stream and day
      Noise      LSQVCE.NOISE              required; the temporal law and its level
     Estimate   "law" | "scale" | "none"  what Options.Estimate="noise" may change: the
```

```
                                        shape and level (default "law"; "scale"
                                        for a per-row sigma), the level only, or
                                        nothing
    ArcFactors logical                   one variance factor per arc (default:
                                        Options.ArcFactors)
    Axes       integer >= 1             axes per epoch, rows epoch-major (default 1)
    Arcs       as addSystem's Arcs       this stream's own schedule (default: the
                                        system's)
  addPrior(sys, name, Name=Value)       a prior group on this system's locals
    On, L, Value, Sigma, Estimate, From  as LSQVCE.TASKDEFINITION/ADDPRIOR, with L
                                      q x nl in the columns task.layout() announces

INPUT   (addSystem's, stored read-only)
  Name      1x1 string     the system's name, unique in the task
  Prepare   function_handle | []   @(day, ctx)
  Settings  1x1 struct     your values; ctx.Settings in Prepare and Build
  Days      1 x D double   the system's calendar ([] = the task's)
  Arcs      [] | double | function_handle   the arc schedule
  Id        [] | number | char   the system's identifier
OUTPUT
  sys       1x1 lsqvce.SystemDefinition   a handle: add-calls change the task

ACCURACY
  Not applicable: bookkeeping (names, scopes, handles); no floating-point
  result. Its compiled system is pinned bitwise against the hand-written
  one in test_taskDefinition.

ERRORS
  lsqvce:TaskDefinition:duplicateName    a local, stream or prior name used twice
  lsqvce:TaskDefinition:badName         a stream name that is not a MATLAB identifier
  lsqvce:TaskDefinition:badValid        Valid is not [] or [start end], start <= end
  lsqvce:TaskDefinition:infoRows         Info has not one row per name
  lsqvce:TaskDefinition:badHandle       Build is not a function handle of (day, ctx)
  lsqvce:TaskDefinition:missingArgument  addStream without Build or Noise; a prior
                                         without Sigma
  lsqvce:TaskDefinition:priorTarget      a prior gives both On and L, or neither
  lsqvce:TaskDefinition:badArcs          (at declaration) some streams have a
                                         schedule and others none
  lsqvce:TaskDefinition:incomplete       (at declaration) a system without a stream,
                                         or without a calendar
  MATLAB:validators:*                    Scope, Estimate, Axes of the wrong kind

EXAMPLE
  task = lsqvce.TaskDefinition(Days=59396:59426);
  task.addCommon(compose("C%d_%d", [2 2 2].', [0 1 2].'));
  grace = task.addSystem("GRACE-FO", Prepare=@prepareGrace, Arcs=4);
  grace.addLocal(["x0" "y0" "z0" "vx0" "vy0" "vz0"], Scope="arc");
  grace.addLocal("accScale", Scope="day");
  grace.addPrior("accScale", On="^accScale", Value=1, Sigma=0.02, Estimate="scale");
  grace.addStream("kbr", Build=@buildKbr, Noise=lsqvce.Noise.white(1e-7));

See also LSQVCE.TASKDEFINITION, LSQVCE.NOISE, LSQVCE.DAYCONTEXT, LSQVCE.DAYDATA
```

8.3 lsqvce.Noise

LSQVCE.NOISE The temporal law and level of one stream's noise.

```
LSQVCE.NOISE  The temporal law and level of one stream's noise.

  n = lsqvce.Noise.white(sigma)            white noise
  n = lsqvce.Noise.arma(ar, sigma)         a known whitening filter, MA = 1
  n = lsqvce.Noise.arma(ar, sigma, MA=ma)
  n = lsqvce.Noise.psd(f, S)               a measured one-sided spectrum, f from 0 Hz
  n = lsqvce.Noise.acf(@rho, sigma)        correlation as a function of the lag
  n = lsqvce.Noise.var(ar, Sigma)          vector AR: axes correlated in time
  n = lsqvce.Noise.custom(@covFcn)         any dense covariance of an arc
  n = lsqvce.Noise.<law>(..., MaxDenseArc=m)

  One constructor per law replaces the struct('type', ...) specification of
  a stream. The object states the LAW and its LEVEL only; what
  Options.Estimate="noise" may change about it is a setting of the stream
  (LSQVCE.SYSTEMDEFINITION/ADDSTREAM: Estimate, ArcFactors). The accuracy
  of each epoch is not part of the law either: Build passes it with each
  arc (LSQVCE.DAYDATA/ADDARC, Sigma=). sigma is always a STANDARD
  DEVIATION, never a variance. Under the stream's Estimate="law" (the
  default) only sigma is used, as the starting level, and the shape is
  estimated from white noise.

  LAWS
    white(sigma)        sigma scalar, or a column of per-row sigmas (a fixed
                        structure: its level can be estimated, its shape not)
    arma(ar, sigma)     ar = [1 a1 ... ap], sigma the innovation standard
                        deviation; MA = [1 b1 ... bq] (default 1), roots strictly
                        inside the unit circle
    psd(f, S)           f (Hz, one-sided, starting at 0) and S (unit^2/Hz),
                        columns of one length; prepend f = 0 when the grid
                        starts higher ([0; f], [S(1); S])
    acf(@rho, sigma)    rho(dt, opt) the correlation at lag dt (seconds)
    var(ar, Sigma)      ar d x d x p, Sigma d x d; no per-arc Sigma with it
    custom(@covFcn)     C = covFcn(t, opt) the covariance of an arc at epochs t
  NAME-VALUE (every law)
    MaxDenseArc  positive integer   the longest arc factorised densely
                                    (default 20 000 epochs; 3.2 GB of factor)
  COST per arc of m rows: white O(m), arma O(m p), var O(m p d^2), psd O(m^3)
  (Toeplitz) or O(m p) with Options.Noise.Whitener = "ar", acf / custom O(m^3).

  INPUT   as the call forms above; every number finite
  OUTPUT  n  1x1 lsqvce.Noise   properties Law (string), Parameters (struct),
                                MaxDenseArc ([] = the default)

  ACCURACY
    Not applicable: it stores a specification; the law's arithmetic is
    LSQVCE.ENGINE.COVMODEL's. Each constructor equals the engines' covariance
    specification field by field (test_taskDefinition).

  ERRORS
    MATLAB:validators:*                 a non-positive or non-finite sigma, a
                                        handle that is not a function handle, a
                                        non-integer MaxDenseArc
```

```
  MATLAB:validation:UnableToConvert   a non-numeric argument
  (the law's own consistency -- spectrum grid, MA invertibility -- is checked
  by LSQVCE.ENGINE.COVMODEL when fit runs, under lsqvce:covModel:*)

EXAMPLE
  lsqvce.Noise.white(1e-7)                       % let the data find the colour
  lsqvce.Noise.arma([1 -1.6 0.9 -0.25], 1e-7)    % a known whitening filter
  lsqvce.Noise.psd([0; 1e-3; 1e-2], [4e-14; 4e-14; 1e-14])

See also LSQVCE.SYSTEMDEFINITION, LSQVCE.TASKDEFINITION, LSQVCE.ENGINE.COVMODEL,
         LSQVCE.ENGINE.CHECKPSD
```

8.4 lsqvce.DayContext

LSQVCE.DAYCONTEXT What Prepare and Build know about the day they serve (read-only).

```
LSQVCE.DAYCONTEXT  What Prepare and Build know about the day they serve (read-only).

  P   = prepare(day, ctx)               a system's Prepare (addSystem)
  out = build(day, ctx)                 a stream's Build (addStream)
  out  = ctx.newDay()                   an empty LSQVCE.DAYDATA for this stream
  cols = ctx.cols(names)                columns of named parameters on this day
  cols = ctx.cols(names, Arc=id)        the same for Scope="arc" locals of arc id

  The toolbox makes one DayContext per call of your Prepare and Build. It
  carries the day, the names of the system and stream, your Settings, the
  result of Prepare, the parameter templates by name, and the arc schedule
  of the day. Build fills ctx.newDay() with arcs (LSQVCE.DAYDATA/ADDARC) and
  returns it.

  COLUMNS BY NAME. Local partials are normally given BY NAME
  (out.addArc(A, l, t, Local={names, P})): the toolbox places them. When the
  partials already form one matrix B (m x ctx.NumLocal), take every column
  from ctx.cols and never compute an offset:
      B = sparse(m, ctx.NumLocal);
      B(:, ctx.cols(["x0" "y0" "z0"], Arc=id)) = Pxyz;
      B(:, ctx.cols("accScale")) = Pacc;
      out.addArc(A, l, t, B=B);
  ctx.cols of common parameters gives columns of A (a Scope="day" common
  moves with the day); of local parameters, columns of B. One call names
  commons or locals, not both.

  PROPERTIES (read-only)
    Day         1x1 double     the day being built (an entry of the calendar)
    System      1x1 string     the system's name
    Stream      1x1 string     the stream's name ("" in Prepare)
    Settings    1x1 struct     the system's Settings
    Prepared    any            what Prepare returned for this day ([] in Prepare,
                               or without Prepare)
    NumCommon   1x1 double     nc, the columns of A
    NumLocal    1x1 double     nl, the columns of B of this system (0 without locals)
    ArcIds      1 x A double   the stream's scheduled arc ids of this day; in
                               Prepare, the system's scheduled arcs of the day ([]
                               when the system declares no schedule: the arcs are
                               what Build adds)
    Common      table          Name, Scope, OnDay, Info of every common template
```

```
  Local      table          Name, Scope, OnDay, Info of every local template
METHODS
  newDay(ctx)                  -> LSQVCE.DAYDATA
  cols(ctx, names, Arc=id)     names 1 x n string -> 1 x n double columns;
                               Arc (a scheduled arc id) is needed by Scope="arc"
                               locals and ignored by the others

INPUT   (made by the toolbox; you never construct one)
OUTPUT  ctx  1x1 lsqvce.DayContext

ACCURACY
  Not applicable: integer bookkeeping over the engines' column plans; the
  columns equal the engines' layout (test_taskDefinition).

ERRORS  (raised while the day is read; fit passes them unchanged)
  lsqvce:DayContext:unknownName   a name that is not a parameter of this task
                                  and system, or not instantiated on this day
  lsqvce:DayContext:arcRequired   a Scope="arc" local asked for without Arc=id,
                                  or with an id the day does not schedule
  lsqvce:DayContext:mixedTables   one cols call names commons and locals
  lsqvce:DayContext:badReturn     Build returned something other than the
                                  LSQVCE.DAYDATA of ctx.newDay()

EXAMPLE
  function out = buildKbr(day, ctx)
      P = ctx.Prepared;                        % what Prepare returned today
      out = ctx.newDay();
      for a = 1:numel(P.arc)
          out.addArc(P.arc(a).A, P.arc(a).omc, P.arc(a).t, ...
                     Local={"kbrBias", ones(numel(P.arc(a).t), 1)});
      end
  end

  See also LSQVCE.DAYDATA, LSQVCE.SYSTEMDEFINITION, LSQVCE.TASKDEFINITION
```

8.5 lsqvce.DayData

LSQVCE.DAYDATA The arcs one Build returns for one stream and one day.

```
LSQVCE.DAYDATA  The arcs one Build returns for one stream and one day.

  out = ctx.newDay()                              made by LSQVCE.DAYCONTEXT only
  out.addArc(A, l, t)                             one arc, no local parameters
  out.addArc(A, l, t, Local={names, P; ...})      local partials by name
  out.addArc(A, l, t, B=B)                        local partials as one m x nl matrix
  out.addArc(..., Sigma=s, Excluded=e, Arc=id)

  Build adds the arcs of its day one by one and returns OUT. Every row is
  RAW: observed minus computed, never whitened or scaled by a weight -- the
  toolbox whitens each arc from the stream's LSQVCE.NOISE and its per-epoch
  Sigma, and eliminates the local parameters arc by arc. An arc may be
  empty (0 rows): a data gap.

  LOCAL PARTIALS, ONE OF TWO WAYS PER ARC
    Local={names, P}   P m x numel(names); several rows {names1, P1; names2, P2}.
                       The scope of each name decides its column: this arc's
```

```
                    block (Scope="arc"), this day's (Scope="day") or the
                    system's (Scope="system"). The usual way.
  B=B               m x ctx.NumLocal (may be sparse), every column taken from
                    ctx.cols(names, Arc=id) -- for partials that already form
                    one matrix. Never computes an offset by hand.
Neither: the arc sees no local parameter (B is all zero).

ARC SCHEDULE. Without a declared schedule (no Arcs on the system or
stream) the arcs are those added, in order. With one, the arcs are
ctx.ArcIds: addArc fills them in order, Arc=id places an arc out of
order, and a scheduled id never added is a data gap.

INPUT (addArc)
  A         m x nc double      design of the common parameters, nc = ctx.NumCommon,
                               columns in addCommon order (Scope="day" commons at
                               ctx.cols(names))
  l         m x 1 double       observed minus computed
  t         m x 1 double       epoch of each row in SECONDS
NAME-VALUE (addArc)
  Local     n x 2 cell         {names, partials} rows (default: none)
  B         m x nl double      the local design as one matrix (default: none)
  Sigma     e x 1 | e x d | e x d x d double   the per-epoch accuracy: standard
                               deviations per epoch, per axis, or covariances
                               (e = m / d epochs); relative to the law's sigma
                               (default: none, D = I)
  Excluded  m x 1 logical      rows of weight 0, zero in A, B and l, kept for their
                               index (default: none)
  Arc       scheduled arc id   place this arc (default: the next scheduled one)
OUTPUT
  out       1x1 lsqvce.DayData   a handle: addArc adds to it

ACCURACY
  Not applicable: it places given numbers into the engines' arc cells
  without arithmetic. Local= and the equivalent B= give the bitwise same
  fit (test_taskDefinition).

ERRORS  (raised while the day is read; fit passes them unchanged)
  lsqvce:DayContext:localAndB     one arc gives both Local and B
  lsqvce:DayContext:badLocal      Local is not an n x 2 cell of {names, P} with P
                                  m x numel(names), or names a column twice
  lsqvce:DayContext:badArc        Arc without a schedule, an id the day does not
                                  schedule, an arc added twice, or more arcs than
                                  scheduled
  lsqvce:DayContext:unknownName   a Local name that is not a local parameter of the
                                  system on this day
  lsqvce:DayContext:arcRequired   an arc-scoped Local without a scheduled arc
  lsqvce:makeLoader:*             the engines' checks of the day (row counts,
                                  Sigma shapes), with day and stream named

EXAMPLE
  out = ctx.newDay();
 out.addArc(A1, l1, t1, Local={["x0" "y0" "z0"], P1; "kbrBias", ones(numel(t1), 1)});
out.addArc(A2, l2, t2, Local={["x0" "y0" "z0"], P2; "kbrBias", ones(numel(t2), 1)}, ...
            Excluded=flagged2);
```

```
  See also LSQVCE.DAYCONTEXT, LSQVCE.SYSTEMDEFINITION, LSQVCE.NOISE
```

8.6 lsqvce.fit

LSQVCE.FIT Fit a least-squares adjustment (optionally with VCE) to declared data.

```
LSQVCE.FIT  Fit a least-squares adjustment (optionally with VCE) to declared data.

  R = lsqvce.fit(task)                       a lsqvce.TaskDefinition (v7, WP-160)
  R = lsqvce.fit(data)                       every default (one GLS solve)
  R = lsqvce.fit(data, opts)                 a lsqvce.Options object
  R = lsqvce.fit(data, Name, Value, ...)     the Options properties, straight on fit
  R = lsqvce.fit(data, opts, Name, Value)    both: the pairs override opts

  THE ONE VERB. You describe the problem with a plain struct `data`
  (the API reference), say what to do with lsqvce.Options, and
  get a lsqvce.Results back:

      R = lsqvce.fit(data, Estimate="noise", Pool="obsType", OnPass=@myInspect);
      R.report;  R.plot("psd");  [C, L] = R.toTable(data);

  THREE DATA KINDS, told apart by the fields of `data`, all on ONE accumulator:
    (A) streaming   data.commonParams, data.days, data.systems(k) with
                    buildDay / covariance / localParams / priors /
                    processingOptions and the optional arc schedule `arcs`
                    (buildDay returns one CELL per arc -- A, B, l, t,
                    Cep as 1 x nArc cells -- and the cells are the schedule
                    unless `arcs` declares one); optional data.priors,
                    data.paramColumns. buildDay is a function handle of
                    each system: several readers in one run are several
                    systems; one reader for several setups takes the setup
                    through processingOptions (its 5th argument) or a
                    closure; data that share no common parameters are
                    separate fit calls. What every stream of a day needs
                    (orbits and partials of a satellite pair) goes in the
                    system's optional prepareDay: called once per build
                    of a day (the current day's result only is kept), its
                    result reaches each buildDay as procOpt.prepared
    (B) in memory   data.A (m x u), data.l (m x 1), optional data.p (m x 1
                    weights); optional data.priors. A FULL m x m SPD weight
                    matrix is kind C's: return it from batchFcn
    (C) batched     data.batchFcn @(k) -> [A, l, p] (or @(k, x) -> [J, w, p] with
                    data.x0 for Gauss-Newton), data.numUnknowns, data.numBatches;
                    optional data.priors
  WHAT RUNS (opts.Estimate / opts.Robust / data.x0):
    Estimate "none"    one generalised solve under the declared laws (adjustBlock)
    Estimate "noise"   the joint variance-component estimator (adjustVCEjoint):
                       one spectral shape per Pool group, one level per stream,
                       per-arc factors when ArcFactors; prior groups with
                       estimate "scale" get their sigma by Helmert / Koch-Kusche,
                       one step per pass from the previous solution, and the
                       loop converges only when that step is below Tolerance too
    Robust "huber" | "igg3" | "bisquare"   IRLS on the same loop: alone it is
                       adjustRobust; with Estimate "noise" the robust pass runs
                       INSIDE the joint loop, R.noise.robust carries
                       the final factors
```

```
  data.x0            Gauss-Newton with damping (adjustNonlinear)
REGULARISATION is declared, never an option: data.priors(j) on the common
parameters and data.systems(k).priors(l) on the local ones -- one struct per
group, by `group` (a name pattern) or `L` (in the columns lsqvce.layout
announces), x0, sigma, from, name, estimate. A free network's datum is the
prior on its null space.
WATCHING A RUN: Options.OnPass fires after every pass with the pass state,
the live accumulator, `data` and every system's processingOptions.

INPUT
  task   1x1 lsqvce.TaskDefinition   the declaration as flat add-calls; fit
                       compiles it with task.declaration() to the struct below
  data   1x1 struct    the declaration (kind A, B or C as above)
  opts   1x1 lsqvce.Options   default lsqvce.Options()
  Name, Value          any property of lsqvce.Options
OUTPUT
  R      1x1 lsqvce.Results   x, sigma0, sol (Cxx, dof, vtv), diag, acc, history,
                              noise (S, Sgroups, C, Cgroups, stream, model, priors,
                              checks -- the model checks, WP-154), stopReason,
                              meta (options as resolved, warnings, seeds, version);
                              methods report, summary, convergence, residuals,
                              contribution, contributions, compare, plot,
                              toTable, coefficients, reliability, systematic,
                              snoop, ellipse, redundancy (LSQVCE.RESULTS)

ACCURACY
  fit adds no arithmetic of its own: the estimate is the block
  accumulator's (LSQVCE.ENGINE.BLOCKACCUMULATOR: the monolithic solve to
  1e-8 relative), the verdict LSQVCE.ENGINE.VERDICT's and the noise
  model LSQVCE.ENGINE.ADJUSTVCEJOINT's -- see theirs. End to end, the
  matrix kind (data.A / .l / .p) reproduces the dense normal equations to
  1e-12 relative (test_fit). WP-138.
  THE FORMAL ERRORS OF AN ESTIMATED NOISE MODEL ARE PLUG-IN (WP-154, M5):
  R.sol.Cxx treats the estimated covariance as known. Measured in the
  review's closed loop (HANDOVER 12.84): their optimism is 3-13 % in sigma
  on short or few arcs (256 epochs, 12 arcs) and ~0 on long records; the
  estimator's own variance adds 4-10 % to x's (eff 1.04-1.10 against the
  oracle that knows the true covariance). Read them as a floor on short
  records; R.noise.checks says when the model's assumptions fail.

ERRORS
  Your declaration is wrong (raised BEFORE any data is read):
    lsqvce:fit:unknownField    a field name the contract does not know
                               (names the field; typos are errors on
                               purpose)
    lsqvce:fit:noSystems       data.systems missing, not a struct, or empty
    lsqvce:fit:noCommon        data.commonParams missing, or not a 2-D
                               cell of template rows
    lsqvce:fit:badScope        a scope-token column 1 does not carry:
                               commonParams needs 'common'|'day' (raised
                               here), localParams needs
                               'arc'|'day'|'system' (raised by makeLoader
                               and relabelled)
    lsqvce:fit:badParamColumns data.paramColumns is malformed: not a
                               struct, an unknown field, only one of
```

```
                              start / end, or a column the table lacks
  lsqvce:fit:missingField     a required system field is absent, or
                              covariance carries no observation type
  lsqvce:fit:noDays           no day list anywhere
  lsqvce:fit:badDays          a day list that is not numeric integers
                              (WP-138; it was noDays)
  lsqvce:fit:duplicateDay     data.days or a system's days names a day
                              twice (it was accumulated twice; WP-135)
  lsqvce:fit:badId            id is not a finite numeric scalar or single
                              char
  lsqvce:fit:mixedIdTypes     the ids mix numbers and characters (an
                              index default is a number; WP-135, its own
                              id since WP-138 -- it was badId)
  lsqvce:fit:duplicateSystem two systems share an id (explicit or
                              index default)
  lsqvce:fit:badValidity      a commonParams validity entry is not [], a
                              fractional MJD or [MJD seconds], or its start
                              is after its end (WP-135; localParams: the
                              makeLoader id, relabelled)
  lsqvce:fit:builderKind      buildDay or prepareDay is not a function
                              handle
  lsqvce:fit:badArcs          arcs is a string (omit arcs: the cells are
                              the arcs), or a
                              resolver returned something that is not
                              unique positive integers
  lsqvce:fit:<makeLoader id> a makeLoader construction error, relabelled
                              (e.g. paramType, badArcs, localPlan);
                              the original is attached as the cause
Your choice is not available here:
  lsqvce:fit:notYet           Robust ~= "none" on a NONLINEAR batch problem
                              (x0): IRLS inside Gauss-Newton is not written;
                              linearise first
  lsqvce:fit:modelKind        Estimate = "noise" on kinds B / C, which have
                              no streams to estimate a law for
  lsqvce:batchSystem:weightMatrix  a batch returned a weight argument that
                              is neither m x 1 nor an m x m SPD matrix. A
                              full SPD P IS accepted on kind C and whitened
                              by its Cholesky factor -- raised
                              by lsqvce.engine.batchSystem
  lsqvce:fit:priorNotActive   (warning) a prior with from > 1 in a
                              single-pass run is not applied (a common
                              or a local group, WP-128), or its from lies
                              beyond the passes the joint loop ran

  lsqvce:fit:priorScaleHeld   (warning) a prior group with estimate
                              "scale" in a run that cannot estimate a
                              prior's sigma (Estimate "none", or the
                              kinds A / B / C) is applied with its
                              declared sigma
  lsqvce:fit:badPrior         data.priors is malformed (unknown
                              field, L not p x nc, sigma not a positive scalar,
                              x0 of the wrong length)
  lsqvce:fit:badEstimate      covariance.<type>.estimate is not
                              "none" | "scale" | "law"
  lsqvce:fit:lawOnFixedStructure  estimate "law" on a stream whose
```

```
                                sigma is per-row (a fixed structure: level only)
    lsqvce:fit:varUnderNoise    a 'var' law under Estimate = "noise": the
                                joint sweep has no vector-AR path
    lsqvce:fit:badMatrix        kind B shapes disagree (A m x u, l m x 1,
                                p m x 1)
    lsqvce:fit:missingField     kind C lacks batchFcn / numBatches, or the
                                linear form lacks numUnknowns
    lsqvce:fit:uCount           kind C carries BOTH x0 and numUnknowns --
                                u is stated once (numel(x0))
  Your buildDay is wrong (raised WHILE READING, by the engine, ids
  unchanged): lsqvce:makeLoader:badDay / dayShape / badArc / buildDay
  (your buildDay threw: the day, the stream and your error as the cause) /
  prepareDay (your prepareDay threw: the day and your error as the cause) /
  badDayCep / axisConflict / cepTwice (the day's Cep and
  axis count), lsqvce:covModel:cepOnVar, lsqvce:accum:arcBadNc / arcBadNl;
  lsqvce:makeLoader:noCep is a once-per-run warning (a multi-axis day
  without Cep is composed with D = I)

See also LSQVCE.OPTIONS, LSQVCE.RESULTS, LSQVCE.LAYOUT, LSQVCE.ENGINE.DECLARATION,
     LSQVCE.ENGINE.MAKELOADER, LSQVCE.ENGINE.ADJUSTBLOCK, LSQVCE.ENGINE.ADJUSTVCEJOINT,
        LSQVCE.ENGINE.ADJUSTROBUST, LSQVCE.ENGINE.ADJUSTNONLINEAR
```

8.7 lsqvce.Options

LSQVCE.OPTIONS The one settings object for LSQVCE.FIT: 12 knobs and 5 groups.

```
LSQVCE.OPTIONS  The one settings object for LSQVCE.FIT: 12 knobs and 5 groups.

  opts = lsqvce.Options()                       every default
  opts = lsqvce.Options(Name, Value, ...)       override some
  opts.Name = Value                             change one later (validated)
  R = lsqvce.fit(data, Name, Value, ...)        the same names, straight on fit

  WHAT TO ESTIMATE, HOW TO RUN. Everything ABOUT A STREAM (its law, whether
  its law / level / arc factors are estimated) is declared on the stream
  (data.systems(k).covariance.<type>: type, sigma, estimate, arcFactors);
  this object holds only RUN policy: 12 knobs plus five typed groups.

  TOP LEVEL
    Estimate   "none" | "noise"   "none": one GLS solve under the declared
                                  laws. "noise": the joint (Ellmer) variance
                                  component estimator on every stream whose
                                  spec allows it (estimate ~= "none"), with
                                  the policy below
    Pool       "stream" | "obsType" | "all"   which streams SHARE one estimated
                                  spectral shape (every stream keeps its own
                                  level regardless); default "stream" (WP-157,
                                  it was "all"): MEASURED in the review's
                                  closed loop, pooling identical spectra gained
                                  nothing in x (eff 1.00-1.08 pooled, 0.90-1.08
                                  per stream), pooling different ones failed
                                  (eff 37 / 132, levels 2-3 x off); pool streams
                                  of one physics deliberately with "obsType"
    ArcFactors logical            per-arc variance factors on (Ellmer 6.5.4),
                                  default true; false = one level per stream.
                                  The factor is the arc's ABSOLUTE Helmert
```

			factor and weights the solve, so sigma0 settles near 1. TO READ AN ARC'S STANDARD DEVIATION back, use all three terms: declared sigma * sqrt(factor) * R.sigma0
Robust	"none" | "huber" | "igg3" | "bisquare"		IRLS on the same loop, every kind (details in RobustOpts). With Estimate "noise" the robust pass runs INSIDE the joint loop: one residual sweep per pass, the factors in every arc's whitener on the next pass; MaxPasses is the budget, RobustOpts.Tuning the constants (RobustOpts.MaxIter / Tol belong to the stand-alone IRLS only)
MaxPasses	double	10	the pass budget of the variance-component loop and of Gauss-Newton. IRLS has its OWN budget, RobustOpts.MaxIter
Tolerance	double	1e-2	the variance-component loop stops when every arc factor changed by less than it, and every spectral bin by less than it times max(1, sqrt(200/r)), r the bin's redundancy: at the default a tenth of the bin's own standard error sqrt(2/r) where that is larger (WP-147: bins the data barely determine need no finer step than their estimate has; R.history.binRatio)
Backend	"qr" | "normal"		square-root vs normal equations
Probes	double	100	Hutchinson probes for the per-lag redundancies; an integer >= 1 (Ellmer's value is 100). The joint estimator has no exact route, so 0 is refused
Seed	[] | double		RNG seed for the probes (recorded)
CacheDays	double	1	built days kept resident per system; Inf = the whole calendar (one build per run). A system's prepareDay result is the current day's only, whatever CacheDays
Verbose	logical | 0 | 1		screen messages: 0 silent, 1 progress. The value is passed to every driver as a flag (logical(Verbose)), so 2 behaves as 1
OnPass	[] | @(ctx)		YOUR inspection hook, called at the end of EVERY pass of every iterative run (the variance-component loop, IRLS, Gauss-Newton) with ONE ctx struct: .driver .pass .x .Cxx .sigma0 .s0sq, the loop state (noise: .S .Sgroups .C .Cgroups .sigma2 .arcKeys .streams .sigma .white .whitePval .maxDev .binRatio .xStep .skips; IRLS: .factor .keys .scale .dx; Gauss-Newton: .vPv .dxRel .lambda), .acc (the LIVE accumulator: call lsqvce.engine.residuals(ctx.acc, ...) or any engine function inside the hook), .data (your declaration as passed to fit) and .processingOptions (per system, as declared). Plot, drawnow, save wherever you like; return nothing to continue, false to stop after this

```
                                pass (stopReason "userStop"). A throwing hook
                                is NOT swallowed (lsqvce:callUser:threw with
                                your error as cause)

GROUPS (structs; set fields one at a time or give a whole struct)
  Noise        Grid 60             spectral bins of the estimated law, 0 ..
                                   Nyquist (an integer >= 2); exact at every
                                   lag for any Grid
               Whitener "toeplitz"|"ar"|"arma"  how a PSD law whitens:
                                                the estimated law and every
                                                declared 'psd' law; under
                                                Estimate "noise" a filter
                                                speeds up the SOLVE only (the
                                                sweep forms each arc's law
                                                densely, maxDenseArc), and
                                                "ar" approximates the
                                                estimated shape
               Smooth 1                         moving average on the PSD,
                                                half-width in bins (WP-154:
                                                the default is 1, it was 0)
                                                -- MEASURED in the review's
                                                closed loop (256-epoch arcs,
                                                AR(1) 0.5..0.99, 10 seeds):
                                                x's variance 1.02-1.11 x the
                                                oracle's (0: 1.10-1.23), the
                                                formal sigma 1.00-1.06 x
                                                (0: 1.16-1.27), the levels
                                                as unbiased, half the passes;
                                                2048-epoch arcs unchanged.
                                                Fewer bins instead (16 / 32)
                                                biased the level low at high
                                                phi (-12 % / -5 % at 0.98)
               FreezeRedundancy 1               components below it hold
               SkipThreshold 0.2                skip RATE of ONE stream's arcs
                                                that refuses the pass
  RobustOpts   Tuning [], MaxIter 30, Tol 1e-8
  Diagnostics  Alpha 1e-3, Bonferroni true, Beta 0.2, RMin 0.01, RWeak 0.1,
               Test "auto"|"pope"|"baarda"|"mad", RobustCheck true,
               Store "all"|"flagged"|"none", NWorst 1000
  Parallel     Mode "serial"|"parfor"|"spmd", DistributeDays false, Threshold 8
  GaussNewton  TolF 1e-10, Lambda0 1e-6     (x0 problems; MaxPasses / Tolerance
               are the budget and the step tolerance)

REGULARISATION is not an option: it is DECLARED on the data, as
pseudo-observations in the user's own columns -- data.priors(k) on the
common parameters (L in the columns of A) and data.systems(k).priors(l) on
the local ones (L in the columns of B), each with x0, sigma, from, group.

METHODS
  engineArgs(target)   the Name-Value list a driver takes (validated)
  toStruct()           every public property as a struct (provenance)
  disp()               grouped display, engine defaults marked *

ACCURACY
  Not applicable: a settings object -- typed knobs, their validation at
```

```
    assignment and the forwarding of their values to the engine
    (engineArgs); it forms no floating-point result. WP-139.

  ERRORS
    MATLAB:validators:*          a typed property receives a value outside
                                 its type / set / range (at assignment)
    lsqvce:Options:badGrid       Noise.Grid is not an integer >= 2
    lsqvce:Options:badVerbose    Verbose is neither logical nor an integer 0..2
    lsqvce:Options:badOnPass     OnPass is neither [] nor a function handle
    lsqvce:Options:badWhitener   Noise.Whitener is not "toeplitz" | "ar" | "arma"
                                 (any case)
    lsqvce:Options:badGroupField a group struct carries an unknown field
    lsqvce:Options:badTarget     engineArgs target is unknown

  EXAMPLE
    opts = lsqvce.Options(Estimate="noise", Pool="obsType");
    opts.Noise.Grid = 2160;  opts.Parallel.Mode = "parfor";
    R = lsqvce.fit(data, opts);
    % Robust ~= "none" together with Estimate = "noise" runs the robust pass
    % inside the joint loop:
    opts.Robust = "huber";

  See also LSQVCE.FIT, LSQVCE.RESULTS, the API reference, RELEASE_NOTES.md
```

8.8 lsqvce.Results

LSQVCE.RESULTS The outcome of a recovery: solution, diagnostics, provenance.

```
LSQVCE.RESULTS  The outcome of a recovery: solution, diagnostics, provenance.
  R = lsqvce.Results(sol, D, acc, diagSol, batchFcn, nBatch, systems)
  R = lsqvce.Results(sol, D, acc, diagSol, batchFcn, nBatch, systems, meta)

  Returned by every driver LSQVCE.FIT dispatches to -- LSQVCE.ENGINE.ADJUSTBLOCK,
  ADJUSTVCEJOINT, ADJUSTROBUST, ADJUSTNONLINEAR. It gathers
  everything a user needs to read and judge an adjustment in one object, and
  keeps the lower layers reachable for experts. You rarely construct it
  yourself; you receive it from lsqvce.fit and call R.report().

  PROPERTIES (read-only)
    x          nc x 1   double   Common-parameter estimate (the field).
    sigma0     1x1      double   A-posteriori variance factor (global).
    stopReason 1x1      string   Why the loop stopped: "single" |
                                 "converged" | "maxPasses" | "userStop" |
                                 "diverged" (Gauss-Newton). Convergence
                                 failure is a RESULT: the last pass is
                                 returned with its covariance and one warning.
    converged  1x1      logical  Did the pipeline converge? (true for a single
                                 solve; set by the iterative engines.)
   diag     1x1    struct  Per-observation diagnostics (LSQVCE.ENGINE.VERDICT.DIAGNOSE):
                                 redundancy, leverage, w-tests, MDB, ...
    history    1xI      struct   Per-iteration record (empty for a single
                                 solve; filled by LSQVCE.ENGINE.ADJUSTVCEJOINT:
                                 .maxDev .binRatio .xStep .S .streams
                                 .sigma .white .whitePval per pass,
                                 .skips, .nIter, and
                                 under Robust the per-pass .robustScale (one
```

```
                              column per stream) .nDown .nZero;
                              by LSQVCE.ENGINE.ADJUSTROBUST: .scale .dx
                              .nDown .nZero and, on the last pass, .factor
                              .keys).
  noise      1x1      struct   Fitted noise models per stream (empty until
                              the variance-component loop populates it):
                              .stream (keys) .model{k} (.sigma the level,
                              .C the law's ACV) .S (the law: the bin
                              spectrum, LSQVCE.KERNEL.HATACV) .Sgroups /
                              .Cgroups (one per pool group)
                              .priors .priorHistory, .checks (the
                              model checks after convergence), and
                              .robust (.influence, .scale per stream,
                              .streams, .factor ONE PER EPOCH in sweep
                              order, .keys, .truncation, .frozen; empty
                              fields under "none").
  sol        1x1      struct   Raw output of acc.solve() (.x, .sigma0, ...).
  acc        1x1      lsqvce.engine.BlockAccumulator   The accumulator (drop here
                              to re-solve, inspect blocks, get residuals).
  source     1x1      struct   Bridged solution from LSQVCE.ENGINE.VERDICT.SOURCE.
 batchFcn  1x1    function_handle  Re-reader used by LSQVCE.ENGINE.VERDICT.DIAGNOSE.
  nBatch     1x1      double   Batch count from the bridge.
  systems    1xM      struct   The resolved system specs (sysID/nl/dayList).
  meta       1x1      struct   Run provenance. Field list is
                              ADD-ONLY across versions, never renamed:
                              .toolboxVersion 1xK char  from Contents.m,
                                              the ONE version source
                              .matlabRelease  1xK char  version('-release')
                              .createdUTC     1xK char  ISO 8601, trailing Z
                              .seeds          1x1 struct every RNG seed the
                                              run resolved (struct() when
                                              none; a resolved-but-unset
                                              seed is stored as [])
                              .options        1x1 struct the RESOLVED
                                              options the run actually used
                              .warnings       every warning identifier
                                              the run raised
                              A bare construction (7 arguments) gets a
                              meta with empty seeds/options -- the field
                              is never missing.

METHODS
  V = R.report(Name,Value,...)   Print/return the level-0..4 verdict
                                 (LSQVCE.REPORT); options pass straight through.
  r = R.redundancy()             Total redundancy sum(r_i) = n - u.
  res = R.residuals(Name,Value)  Whitened/raw residuals (acc.residuals).
  C = R.contributions()          Every (system, day)'s reduced contribution under
                                 the run's final weights, for
                                 LSQVCE.SOLVECONTRIBUTIONS (a bootstrap over days).
  con = R.contribution(...)      Per-group information share.
  T = R.convergence(Name,Value)  Per-ITERATION trajectory: when the loop
                                 settled (needs .history).
  F = R.compare(Rred)            Nested-model F test against a reduced fit.
  S = R.systematic(Name,Value)   Unmodelled signal in the residuals: Ljung-Box,
                                 runs, binned means.
```

```
  [D, REJ, sol] = R.snoop(...)   Iterative data snooping; R is not changed.
  T = R.reliability()            Per-observation r, w, MDB and (w, r) quadrant.
  E = R.ellipse(i, j, ...)       Confidence ellipse of the parameter pair (i, j).
  fh = R.plot(kind)             "residuals" | "contribution" | "psd" | "convergence"
                                 (one figure per question, nothing exported).
 T = R.toTable(task)         One table of every parameter of a lsqvce.TaskDefinition:
                                 System, Name, Scope, Day, Arc, Estimate, Std
  [C, L, map] = R.toTable(data)  The expanded parameter tables with estimate and
                                 standard deviation written in.
  T = R.summary()                Per-STREAM final state: sigma, AR order,
                                 whiteness (needs .history).
  T = R.coefficients(Alpha=a)    The estimates as a table with formal
                                 errors and (1-a) confidence bounds
                                 (default a = 0.05).
  disp(R)                        One-screen summary.

THE TWO ITERATION READERS ARE COMPLEMENTARY. convergence() reduces across
STREAMS (one row per pass); summary() reduces across PASSES (one row per
stream). A converged loop can still hide a single stream that never
whitened -- the per-pass view reports the worst p-value without naming it.

EXAMPLE
  R = lsqvce.fit(data, Estimate="noise");
  R.report();
  if ~R.converged, lsqvce.util.log.warn('did not converge'); end
  x = R.x;   s0 = R.sigma0;

ERRORS
  lsqvce:Results:noVPv          compare() on a Results without a finite v'Pv
  lsqvce:Results:noHistory      convergence() / summary() / plot("convergence")
                                on a Results without the noise loop's
                                per-stream history: a single solve, or a
                                stand-alone Robust / Nonlinear run (their
                                passes are R.history, one record each --
                                WP-135); use Estimate = "noise"
  lsqvce:Results:noCovariance   coefficients() on a Results whose sol
                                carries no Cxx
  lsqvce:Results:noAccumulator  residuals()/contribution() on a Results
                                without an accumulator (fit kinds B / C)
  lsqvce:Results:noPerObservation reliability() on a fit whose diagnostics
                                kept no per-observation vectors
  lsqvce:Results:badPlotKind    plot() with an unknown kind, or a Stream
                                that matches no stream
  lsqvce:Results:tableMismatch  toTable() with a declaration that is not
                                the one this Results came from
  lsqvce:Results:compareEstimatedWeights  compare() on fits whose weights
                                were estimated (one fixed weighting needed)
  lsqvce:Results:contributionsNonlinear   contributions() of a
                                Gauss-Newton run
  lsqvce:Results:contributionsInconsistent contributions() whose days'
                                dof do not add up to the run's

ACCURACY
  Results adds no numerics: x and sigma0 are the engine's (sigma0 is the
  A-POSTERIORI std of unit weight; sol.Cxx the covariance scaled by it --
```

```
   the streaming contract every producer must establish; a v'Pv the
   backend refused as cancelled is the verdict's residual pass,
   LSQVCE.ENGINE.VERDICT.SUPERSEDE),
   report() is lsqvce.report on .diag, coefficients() is arithmetic on x
   and diag(Cxx) with z(0.05) = 1.959963984540054 in closed form.

 See also LSQVCE.FIT, LSQVCE.ENGINE.ADJUSTBLOCK, LSQVCE.ENGINE.ADJUSTVCEJOINT, LSQVCE.REPORT,
          LSQVCE.ENGINE.VERDICT, LSQVCE.ENGINE.BLOCKACCUMULATOR
```

8.9 lsqvce.report

LSQVCE.REPORT Judge a least-squares adjustment and say so in words.

```
LSQVCE.REPORT  Judge a least-squares adjustment and say so in words.

  V = LSQVCE.REPORT(D) takes the struct from lsqvce.engine.verdict.diagnose and prints a
   verdict. It returns a struct of machine-readable findings so the same logic
   can drive an automated pipeline.

   The report is ordered so that a failure at one level invalidates the levels
   below it. It refuses to report parameter precision for an adjustment whose
   integrity checks failed, because a sigma computed from a broken system is
   not a conservative estimate -- it is a fabrication.

   LEVELS
     0  INTEGRITY   sum(r) == n-u, vPv >= 0, vPv consistent between passes.
                    These test the CODE, not the data. Failure here means
                    every number below is meaningless.
     1  GLOBAL      variance factor vs chi-square.
     2  LOCALISE    outliers (data snooping), with Bonferroni.
     3  RELIABILITY redundancy numbers, MDB, external reliability.
                    This is the level that says which observations you are
                    NOT ABLE TO CHECK.
     4  PARAMETERS  correlations, conditioning, precision.

   NAME-VALUE
     'Verbose'  true  print the report (default true)
     'Width'    78    line width (>= 40)
     'Provenance' struct()  a run-provenance struct (LSQVCE.RESULTS .meta);
                         when non-empty, ONE level-0 NOTE line
                         (code 'PROVENANCE') states toolbox version, MATLAB
                         release, and creation time. R.report() forwards it
                         automatically; a bare lsqvce.report(D) prints none.
     'Iteration'  struct()  the loop state of an ITERATIVE run:
                         .converged, .stopReason, .nIter and optionally .loop
                         (which loop iterated: 'noise' -- the default, the
                         stochastic model -- 'irls' or 'gaussNewton'; WP-138:
                         a code, it was the display label .what; another value
                         is refused, lsqvce:report:badIteration). When non-empty and the
                         run was not a single solve, ONE level-1 line says
                         whether the stochastic model settled -- 'NOISE_SETTLED'
                         (NOTE) or 'NOISE_UNSETTLED' (WARN). Every number from
                         level 1 down rests on those weights, so a model that was
                         still moving qualifies all of them. R.report() forwards
                         it automatically. An optional .influence (char) names
                         the robust influence of an IRLS run; when non-empty ONE
```

```
                    level-1 NOTE ('ROBUST_FIT') says sigma0 and the
                    covariance are those of the reweighted problem.

  OUTPUT V
    V.verdict   'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
    V.findings  cell array of struct(level, severity, code, msg)
    V.message   the one-paragraph explanation printed under the verdict
    V.nError, V.nWarn, V.nNote
    V.integrityOK  true when every level-0 check held

  INPUT
    D         1x1 struct  diagnostics from LSQVCE.ENGINE.VERDICT.DIAGNOSE
    Name/Value: 'Verbose', 'Width', 'Provenance', 'Iteration' (above).
              The test settings (alpha, rWeak, ...) are not report options:
              they shape D, through lsqvce.Options' Diagnostics group.

  EXAMPLE
      data = struct('batchFcn', src, 'numUnknowns', u, 'numBatches', 4);
      R = lsqvce.fit(data, Verbose=false);
      V = lsqvce.report(R.diag);         % prints the five levels (= R.report)

      % V.verdict    'PASS' | 'PASS WITH WARNINGS' | 'SUSPECT' | 'INVALID'
      % V.findings   machine-readable: {level, severity, code, msg}

      % Drive a pipeline off the codes rather than the text:
      codes = cellfun(@(f) f.code, V.findings, 'UniformOutput', false);
      if ismember('SNOOP_CONTAMINATED', codes)
          error('Pope is blind here -- re-run with Diagnostics.Test = "mad".');
      end

  ACCURACY
    Not applicable to the verdict: it reads D (LSQVCE.ENGINE.VERDICT.DIAGNOSE
    states the accuracy of every number it prints) and forms one band of
    its own, the variance factor's chi2(r) / r at 0.5 % / 99.5 %, by
    LSQVCE.KERNEL.STAT.CHI2INV (|F(x) - p| / p < 1e-11 for r <= 1e7; its
    own ACCURACY). WP-138.

  ERRORS
    lsqvce:report:badIteration  Iteration.loop names no loop the report
                                knows ('noise', 'irls', 'gaussNewton'), or
                                Iteration carries the retired .what

  See also LSQVCE.ENGINE.VERDICT.DIAGNOSE, LSQVCE.ENGINE.VERDICT.SYSTEMATIC.
```

8.10 lsqvce.layout

LSQVCE.LAYOUT The column layout fit will use – so you can declare priors in it.

```
LSQVCE.LAYOUT  The column layout fit will use -- so you can declare priors in it.
  lay = lsqvce.layout(data)

  The toolbox derives from your parameter tables (scope tokens, calendar,
  arcs, validity) which column of A and of every system's B belongs to
  which instantiated parameter. This function announces that layout BEFORE
  fit runs, so that data.priors(j).L (columns of A) and
  data.systems(k).priors(l).L (columns of B) can be written in it -- the
```

```
only knowledge the user's regularisation and the toolbox share.

lay = lsqvce.layout(task)   a lsqvce.TaskDefinition: the same layout, plus
                            .common(j).name, .systems(k).name and
                            .systems(k).local(c).name

INPUT   data  1x1 struct  a kind-A declaration (the API reference), or a
                          1x1 lsqvce.TaskDefinition
OUTPUT  lay   1x1 struct
  .nc            1x1        columns of A
 .common      nc x 1 struct (.row .day)    the template row and (for 'day' rows) the day of every col
  .systems(k)    1x1 struct per system:
     .nl         1x1        columns of B
   .local    nl x 1 struct (.row .day .arc) the template row / day / arc of every column of B
    .col      @(row, day, arc) -> column    lookup: the column of template row `row` on
                                       `day` (and `arc` for 'arc' rows; [] otherwise);
                                            0 when that instance does not exist
 .colCommon    @(row, day) -> column        the same for A ([] day for 'common' rows)

EXAMPLE  (a Tikhonov prior on local row 7 of system 1, on every day)
  lay = lsqvce.layout(data);
  nl  = lay.systems(1).nl;  rows = zeros(0, nl);
  for d = data.days
    c = lay.systems(1).col(7, d, []);  r = zeros(1, nl);  r(c) = 1;  rows(end+1, :) = r;
  end
 data.systems(1).priors = struct('L', rows, 'x0', 0, 'sigma', 0.01, 'name', 'xaccscale', 'estimate

ACCURACY
  Not applicable: it counts and indexes columns -- integers from the
  declaration's plans, the very plans fit builds (declaration); no
  floating-point result. test_fit pins the columns
  (testLayoutOfASystemWithoutLocals,
  testSystemWithoutDaysClaimsOnlyTheGlobalCalendar). WP-139.

See also LSQVCE.FIT, LSQVCE.RESULTS/TOTABLE, the API reference
```

8.11 lsqvce.solveContributions

LSQVCE.SOLVECONTRIBUTIONS Solve a weighted combination of a run’s days without re-reading them.

```
LSQVCE.SOLVECONTRIBUTIONS  Solve a weighted combination of a run's days without re-
reading them.

  out = LSQVCE.SOLVECONTRIBUTIONS(C, c)
  out = LSQVCE.SOLVECONTRIBUTIONS(C, c, 'Locals', true)

  For a bootstrap over days (WP-158, issue #183). C = R.contributions()
  holds every (system, day)'s reduced contribution of a fit; c gives each
  day a multiplicity -- how often a replicate drew it (0, 1, 2, ...), or any
  non-negative real for a Bayesian bootstrap. The combination's reduced
  normal equations are the weighted sum of the days',
      N* = sum_d c_d N_d,  n* = sum_d c_d n_d,  l'l* = sum_d c_d l'l_d,
      dof* = sum_d c_d dof_d,
  formed on the square root (each day's rows times sqrt(c_d)) on the QR
  backend, and solved exactly as fit solves its accumulation: the common
```

```
prior added ONCE, the same rank rule and cancellation guards. A day drawn
twice contributes 2 N_d -- no relabelling around lsqvce:fit:duplicateDay.
One replicate costs O(D nc^2) for D days and nc common parameters,
independent of the number of observations per day.

THE WEIGHTS ARE FIXED. Every day carries the run's final weights: a noise
run's estimated variance components and a robust run's final (hard)
weights are NOT re-estimated per combination -- a bootstrap of the
estimator under fixed weights. out.weights says which weights those
were. A bootstrap that re-estimates them needs a full fit per replicate.
A robust run's sigma0 is divided by its truncation factor tau (C.truncation)
as fit's hard solution is: every combination of days is a sample of kept
epochs truncated alike.

INPUT
  C   1x1 struct   R.contributions() of a linear fit
  c   1 x D double multiplicity per day, aligned to C.days; finite, >= 0,
                   at least one day > 0

OPTIONS (Name/Value)
  'Locals'  logical  also back-substitute the local parameters of every
                     day drawn (c_d > 0), with their covariance (default false)

OUTPUT
  out.x       nc x 1   the common parameters
  out.Cxx     nc x nc  their covariance (s0sq times the cofactor)
  out.sigma0  1x1      sqrt(s0sq);  out.s0sq, out.dof, out.vtv as fit's solve
  out.weights 1x1      C.weights: .estimate, .robust, .fixed = true
  out.locals  struct array, with 'Locals' true, one per system and drawn
              day: .sysID, .day, .multiplicity, .localIdx (the system's
              local indices), .y, .Cyy; [] otherwise

EXAMPLE
  R = lsqvce.fit(data, opts);
  C = R.contributions();                        % one data pass
  D = numel(C.days);  B = 100;  X = zeros(numel(R.x), B);
  for b = 1:B
      draw = randi(D, 1, D);                    % D days with replacement
      c = accumarray(draw(:), 1, [D 1]).';
      X(:, b) = lsqvce.solveContributions(C, c).x;
  end
  Cboot = cov(X.');                             % the bootstrap covariance of x

ACCURACY
  With every multiplicity 1 the combination reproduces fit's x, Cxx and
  sigma0 on the QR backend to 1e-8 relative, the issue's acceptance
  bound (measured 2e-10 in x and Cxx, 2e-14 in sigma0 at cond(A) ~ 9e4:
  the days' reduced rows are stacked and re-factorised where fit stacked
  each system's); with a multiplicity vector it equals a full fit over the
  expanded day list (a day drawn twice entered twice, under the same
  fixed weights) to 1e-8 (measured 9e-10); the locals of a drawn day equal
  fit's back-substitution to 1e-8 (measured 2e-10); a noise run and a
  robust run (sigma0 with tau) reproduce their fit alike (test_fit
  (testContributionsReproduceTheFit, testContributionsEqualAFitOverTheExpandedDays,
 testContributionsReturnTheDrawnDaysLocals, testContributionsOfANoiseAndARobustRun)).
```

```
    On the normal backend the agreement is that of the normal equations
    themselves, cond(N) eps: at cond(N) ~ 7e9 fit's own normal solve is
    5e-4 from the QR one (testContributionsOnTheNormalBackend uses a
    well-conditioned fixture).

  ERRORS
    lsqvce:solveContributions:badContributions  C is not R.contributions()'s struct
    lsqvce:solveContributions:badMultiplicity   c is not 1 x numel(C.days), finite,
                                                >= 0 with one day > 0
    (and the solve's own: lsqvce:accum:priorNotPD, lsqvce:qr:*)

  See also LSQVCE.FIT, LSQVCE.RESULTS
```

9 The engine, kernel and util tiers

What `fit` dispatches to and what it is built from: one line per public unit, from the `Contents.m` files (generated). `doc lsqvce.engine.<name>` opens the full page. Calls go downward only: the user tier calls the engine, the engine calls kernel and util.

9.1 lsqvce.engine — drivers, accumulator, loaders, sweeps, diagnostics

unit	what
`adjustNonlinear`	Gauss-Newton / Levenberg as a policy on the block path
`batchSystem`	a batch source as one local-free streaming system
`adjustRobust`	IRLS (Huber/IGG3/bisquare) as a policy on the block path
`adjustBlock`	one GLS solve over the streaming block-arrow pipeline
`adjustVCEjoint`	Ellmer ten-step joint co-estimation with keyed arc factors
`applyRegularisation`	regularisation of the reduced system
`arcVarianceFactors`	per-arc variance factors
`BlockAccumulator`	block-arrow accumulation with per-system Schur pre-elimination
`BlockCofactor`	the cofactor of the full parameter set, kept as blocks
`checkPSD`	PSD sanity check
`collectResiduals`	residual collection per stream
`CommonPrior`	prior on the common block (regularisation as observations)
`contribution`	information contribution per group
`covComponent`	covariance component estimation
`covModel`	covariance spec -> whitening operators (per arc)
`datum`	datum defect detection and handling
`declaration`	validate a kind-A data struct and build its loaders
`gcvLambda`	GCV choice of the regularisation weight
`jointModelOpt`	joint-mode model -> loader UserOpt translation
`localDiagnostics`	diagnostics of the local blocks

unit	what
localColumns	which (row, day, arc) owns each column of B
makeLoader	day-builder -> Loader (built inside fit from the data struct)
mapreduce	serial / parfor / spmd task runner with ordered reduce
NormalEqSystem	normal-equation backend
outliers	outlier listing from diagnostics
psdVCE	PSD amplitude + arc-factor VCE (Ellmer workhorse)
QRSystem	QR backend
ReducedContribution	reduced-normal contribution bookkeeping
redundancy	redundancy numbers (data pass)
residuals	second data pass: residuals and quadratic forms
resolveNc	common-parameter count derived from the systems
synthLoader	synthetic block-arrow loader (engine-tier fixture; leaves in M4)
sharedProbe	shared Hutchinson probe generator (RNG contract)
streamContribution	contribution per stream
traceEstimate	stochastic trace estimate
vce	Helmert group variance components (exact
verdict	the verdict: diagnostics, snooping, systematics, F test, ellipse
runMeta	immutable provenance record for Results.meta

9.2 lsqvce.kernel — the numerical kernels

unit	what
ar2arma	AR -> ARMA conversion
armaACV	ARMA autocovariance
armaWhiten	ARMA whitening filter (batched)
crossCorr	cross-correlation of residual streams
dctI	DCT-I transform
decorrFactor	decorrelation factor of a dense covariance
hatACV	ACV of every bin of a piecewise-linear spectrum
levinsonDurbin	Levinson-Durbin recursion
psd2acv	PSD -> autocovariance
lawInverse	explicit inverse of an arc's covariance law
redundancyLags	redundancy per lag without the dense W^-1
truncationDeficit	the redundancy a robust sample loses to its truncation
wSnoop	Baarda's w per epoch with iterative data snooping
separableWhiten	separable (temporal x axis) whitening
stat	distributions and test thresholds, base MATLAB only
toeplitzCofactor	Toeplitz cofactor construction
varACV	VAR autocovariance
varWhiten	VAR whitening

unit	what
`whitenessCorr`	whiteness (Ljung-Box) statistic
`whittle`	multichannel Levinson (Whittle) VAR fit
`robustWeights`	IRLS weight factors from standardised residuals

9.3 lsqvce.util — plumbing

unit	what
`callUser`	call a user function: outputs counted once, errors wrapped
`getfielddef`	struct field with default
`toolboxVersion`	the version string from Contents.m (one source)
`onPass`	fire the per-pass inspection hook (Options.OnPass)
`rowWeightKey`	the one key convention for per-arc robust weights
`streamKey`	the one stream key ’satKey
`priorStream`	the prior pseudo-streams’ obsType, made and recognised once
`arcKey`	the one arc key of the joint estimator
`log`	toolbox-wide logging state
`mergeStruct`	struct merge (later wins)
`verbosity`	verbosity level resolution and formatting

10 Error and warning identifiers

Every identifier the package raises (generated from the identifier catalogue, which is kept closed both ways: nothing is raised that is not listed, nothing is listed that is not raised). One family is not listed by name: an `lsqvce:makeLoader:<reason>` raised while `fit` builds a system’s loader reaches you as `lsqvce:fit:<reason>`, with the original attached as the cause.

identifier	raised by	kind	introduced
`lsqvce:BlockCofactor:badBlocks`	BlockCofactor	error	v6.3.0
`lsqvce:BlockCofactor:multiSystemBlocks`	BlockCofactor	error	v6.3.0
`lsqvce:Options:badGrid`	Options	error	v4.0
`lsqvce:Options:badGroupField`	Options	error	v4.0
`lsqvce:Options:badOnPass`	Options	error	v4.0
`lsqvce:Options:badTarget`	Options	error	v4.0
`lsqvce:Options:badVerbose`	Options	error	v4.0
`lsqvce:Options:badWhitener`	Options	error	v6.22.0
`lsqvce:Results:badPlotKind`	Results	error	v4.0
`lsqvce:Results:contributionsInconsistent`	Results	error	v6.35.0
`lsqvce:Results:contributionsNonlinear`	Results	error	v6.35.0
`lsqvce:Results:noAccumulator`	Results	error	v4.0
`lsqvce:Results:noCovariance`	Results	error	v4.0
`lsqvce:Results:noHistory`	Results	error	v4.0
`lsqvce:Results:noPerObservation`	Results	error	v6.5.0
`lsqvce:Results:noVPv`	Results	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:Results:compareMismatch`	`Results.unitedWeights`	error	v6.28.0
`lsqvce:Results:tableMismatch`	`Results`	error	v4.0
`lsqvce:accum:dayLocalsRangeDefect`	`engine.BlockAccumulator`	error	v6.35.0
`lsqvce:accum:localsCouplingDays`	`engine.BlockAccumulator`	error	v6.35.0
`lsqvce:accum:noContributionsToCore`	`engine.BlockAccumulator`	error	v6.35.0
`lsqvce:accum:regularisedGram`	`engine.BlockAccumulator`	error	v6.35.0
`lsqvce:lawInverse:notPd`	`kernel.lawInverse`	error	v6.34.0
`lsqvce:lawInverse:notSquare`	`kernel.lawInverse`	error	v6.34.0
`lsqvce:qr:droppedNonzero`	`engine.QRSystem`	error	v6.35.0
`lsqvce:report:badIteration`	`report`	error	v6.24.0
`lsqvce:accum:arcBadNc`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:arcBadNl`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:arcNotFinite`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:arcRowMismatch`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badArc`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badCxx`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badLoader`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badMeta`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badPrior`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badSpecs`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badThresholds`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:badX`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:blockSchurFailed`	`engine.BlockAccumulator`	warning	v4.0
`lsqvce:accum:cancellation`	`engine.BlockAccumulator`	warning	v4.0
`lsqvce:accum:duplicateSystem`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:localNotRedundant`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:localSummaryShapes`	`engine.BlockAccumulator`	warning	v4.0
`lsqvce:accum:metaMismatch`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:noArcs`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:noDays`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:noStore`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:noSystems`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:notWholeEngine`	`engine.private.arcBatchWriter`	error	v4.0
`lsqvce:accum:pinvFallback`	`engine.BlockAccumulator`	warning	v4.0
`lsqvce:accum:priorNotPd`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:priorSize`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:accum:unknownSystem`	`engine.BlockAccumulator`	error	v4.0
`lsqvce:adjustBlock:badCol`	`engine.private.normalizeSystems`	error	v4.0
`lsqvce:adjustBlock:badLength`	`engine.private.normalizeSystems`	error	v4.0
`lsqvce:adjustBlock:badShape`	`engine.private.normalizeSystems`	error	v4.0
`lsqvce:adjustBlock:badSystem`	`engine.private.normalizeSystems`	error	v4.0
`lsqvce:adjustBlock:noDays`	`engine.private.normalizeSystems`	error	v4.0
`lsqvce:adjustNonlinear:badX0`	`kernel.adjustNonlinear`	error	v4.0
`lsqvce:adjustNonlinear:diverged`	`kernel.adjustNonlinear`	warning	v4.0
`lsqvce:adjustNonlinear:maxIter`	`kernel.adjustNonlinear`	warning	v4.0
`lsqvce:adjustNonlinear:nonFinite`	`kernel.adjustNonlinear`	error	v4.0
`lsqvce:adjustRobust:collapsed`	`kernel.adjustRobust`	error	v4.0
`lsqvce:adjustRobust:notConverged`	`kernel.adjustRobust`	warning	v4.0
`lsqvce:adjustVCEjoint:arcMismatch`	`kernel.adjustVCEjoint`	error	v4.0
`lsqvce:adjustVCEjoint:badAxis`	`kernel.adjustVCEjoint`	error	v4.0
`lsqvce:adjustVCEjoint:badS0`	`kernel.adjustVCEjoint`	error	v6.7.0
`lsqvce:adjustVCEjoint:badSigma2`	`kernel.adjustVCEjoint`	error	v4.0
`lsqvce:adjustVCEjoint:localNotPd`	`kernel.adjustVCEjoint`	error	v4.0
`lsqvce:adjustVCEjoint:negDof`	`kernel.adjustVCEjoint`	error	v4.0

identifier	raised by	kind	introduced
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	error	v4.0
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	error	v6.1
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	warning	v4.0
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	warning	v5.3
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	error	v4.0
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	error	v4.0
lsqvce:adjustVCEjoint:level	engine.adjustVCEjoint	warning	v6.32.0
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	warning	v6.32.0
lsqvce:adjustVCEjoint:engine	engine.adjustVCEjoint	warning	v6.32.0
lsqvce:applyReg:badLambda	engine.applyRegularisation	error	v4.0
lsqvce:applyReg:badRegMatrix	engine.applyRegularisation	error	v4.0
lsqvce:applyReg:notPD	engine.applyRegularisation	error	v4.0
lsqvce:applyReg:sizeMismatch	engine.applyRegularisation	error	v4.0
lsqvce:applyReg:unknown	engine.applyRegularisation	error	v4.0
lsqvce:ar2arma:degenerate	kernel.ar2arma	error	v4.0
lsqvce:ar2arma:degenerateMA	kernel.ar2arma	warning	v4.0
lsqvce:ar2arma:nonInvertibleMA	kernel.ar2arma	warning	v6.14.0
lsqvce:ar2arma:tooShort	kernel.ar2arma	error	v4.0
lsqvce:ar2arma:unstable	kernel.ar2arma	warning	v4.0
lsqvce:arcVarianceFactors:engineArchMismatch	engine.arcVarianceFactors	error	v4.0
lsqvce:arcVarianceFactors:engineBadMeanV	engine.arcVarianceFactors	warning	v4.0
lsqvce:arcVarianceFactors:engineBadOmegaV	engine.arcVarianceFactors	warning	v4.0
lsqvce:armaACV:nonStationary	kernel.armaACV	error	v4.0
lsqvce:armaACV:zeroLeakingAR	kernel.armaACV	error	v4.0
lsqvce:armaWhiten:acvTooShort	kernel.armaWhiten	error	v4.0
lsqvce:armaWhiten:badSigma	kernel.armaWhiten	error	v4.0
lsqvce:armaWhiten:sizeMismatch	kernel.armaWhiten	error	v4.0
lsqvce:armaWhiten:starNotPD	kernel.armaWhiten	warning	v4.0
lsqvce:batchSystem:badMatrix	engine.batchSystem	error	v4.0
lsqvce:batchSystem:badSigma	engine.batchSystem	error	v4.0
lsqvce:batchSystem:weightMismatch	engine.batchSystem	error	v4.0
lsqvce:callUser:badHandle	util.callUser	error	v4.0
lsqvce:callUser:threw	util.callUser	error	v4.0
lsqvce:callUser:tooFew	util.callUser	error	v4.0
lsqvce:checkPSD:badFit	engine.checkPSD	warning	v4.0
lsqvce:checkPSD:badModel	engine.checkPSD	error	v4.0
lsqvce:checkPSD:marginalFit	engine.checkPSD	warning	v4.0
lsqvce:checkPSD:unstable	engine.checkPSD	warning	v4.0
lsqvce:collectResiduals:badRes	engine.collectResiduals	error	v4.0
lsqvce:covComponent:badLag	engine.covComponent	error	v4.0
lsqvce:covComponent:lagTooLarge	engine.covComponent	error	v4.0
lsqvce:covModel:badCep	engine.covModel	error	v4.0
lsqvce:covModel:badDeleteRows	engine.covModel	error	v6.31.0
lsqvce:covModel:badPsdMode	engine.covModel	error	v4.0
lsqvce:covModel:badSigma	engine.covModel	error	v4.0
lsqvce:covModel:badWhitenMode	engine.covModel	error	v4.0
lsqvce:covModel:cepOnVar	engine.covModel	error	v5.3
lsqvce:covModel:denseAcfTooLong	engine.covModel	error	v6.14.0
lsqvce:covModel:degenerateACF	engine.covModel	error	v4.0
lsqvce:covModel:maNotInvertible	engine.covModel	error	v6.13.0
lsqvce:covModel:negativePSD	engine.covModel	warning	v4.0
lsqvce:covModel:noACF	engine.covModel	error	v4.0
lsqvce:covModel:noAR	engine.covModel	error	v4.0
lsqvce:covModel:noCovForm	engine.covModel	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:covModel:noFcn`	engine.covModel	error	v4.0
`lsqvce:covModel:noFilterForm`	engine.covModel	error	v4.0
`lsqvce:covModel:noPSD`	engine.covModel	error	v4.0
`lsqvce:covModel:noSigma`	engine.covModel	error	v4.0
`lsqvce:covModel:noType`	engine.covModel	error	v4.0
`lsqvce:covModel:noVAR`	engine.covModel	error	v4.0
`lsqvce:covModel:notWhoseArcs`	engine.covModel	error	v4.0
`lsqvce:covModel:psdSanityInterval`	engine.covModel	error	v4.0
`lsqvce:covModel:sigmaLength`	engine.covModel	error	v4.0
`lsqvce:covModel:unknownType`	engine.covModel	error	v4.0
`lsqvce:crossCorr:shortSeries`	kernel.crossCorr	warning	v4.0
`lsqvce:datum:BnotNull`	engine.datum	warning	v4.0
`lsqvce:datum:autoDetected`	engine.datum	warning	v4.0
`lsqvce:datum:badB`	engine.datum	error	v4.0
`lsqvce:datum:rankDeficientB`	engine.datum	error	v4.0
`lsqvce:datum:zeroN`	engine.datum	error	v4.0
`lsqvce:dctI:tooShort`	kernel.dctI	error	v4.0
`lsqvce:decorrFactor:noPD`	kernel.decorrFactor	mixed	v4.0
`lsqvce:diagnose:badPmatrix`	engine.verdict	error	v4.0
`lsqvce:diagnose:batchMismatch`	engine.verdict	error	v4.0
`lsqvce:diagnose:nBatches`	engine.verdict	error	v4.0
`lsqvce:diagnose:noRobustScale`	engine.verdict	error	v4.0
`lsqvce:diagnoseAccum:nanKeepLevel`	engine.verdict	error	v4.0
`lsqvce:diagnoseAccum:noSystems`	engine.verdict	error	v4.0
`lsqvce:ellipse:badPair`	engine.verdict	error	v4.0
`lsqvce:ellipse:samePair`	engine.verdict	error	v4.0
`lsqvce:fit:badArcs`	engine.declaration	error	v4.0
`lsqvce:fit:badEstimate`	fit	error	v4.0
`lsqvce:fit:badId`	engine.declaration	error	v4.0
`lsqvce:fit:badMatrix`	fit	error	v4.0
`lsqvce:fit:badParamColumns`	engine.declaration	error	v4.0
`lsqvce:fit:badPrior`	fit	error	v4.0
`lsqvce:fit:badScope`	engine.declaration	error	v4.0
`lsqvce:fit:builderKind`	engine.declaration	error	v4.0
`lsqvce:fit:duplicateSystem`	engine.declaration	error	v4.0
`lsqvce:fit:lawOnFixedSlot`	fit	error	v4.0
`lsqvce:fit:missingField`	engine.declaration, fit	error	v4.0
`lsqvce:fit:modelKind`	fit	error	v4.0
`lsqvce:fit:noCommon`	engine.declaration	error	v4.0
`lsqvce:fit:noDays`	engine.declaration	error	v4.0
`lsqvce:fit:noSystems`	engine.declaration	error	v4.0
`lsqvce:fit:notYet`	fit	error	v4.0
`lsqvce:fit:priorNotActive`	fit	warning	v4.0
`lsqvce:fit:priorScaleHeld`	fit	warning	v6.28.0
`lsqvce:fit:uCount`	fit	error	v4.0
`lsqvce:fit:varUnderNoise`	fit	error	v6.1.5
`lsqvce:fit:unknownField`	engine.declaration, fit	error	v4.0
`lsqvce:fit:badValidity`	engine.declaration	error	v6.22.0
`lsqvce:fit:duplicateDay`	engine.declaration	error	v6.22.0
`lsqvce:fit:badDays`	engine.declaration	error	v6.24.0
`lsqvce:fit:mixedIdTypes`	engine.declaration	error	v6.24.0
`lsqvce:ftest:differentData`	engine.verdict	error	v4.0
`lsqvce:ftest:invalidPremise`	engine.verdict	warning	v4.0
`lsqvce:ftest:noRedundancy`	engine.verdict	error	v4.0

identifier	raised by	kind	introduced
`lsqvce:ftest:notNested`	engine.verdict	error	v4.0
`lsqvce:ftest:zeroVariance`	engine.verdict	error	v4.0
`lsqvce:gcv:atLowerBound`	engine.gcvLambda	warning	v4.0
`lsqvce:gcv:atUpperBound`	engine.gcvLambda	warning	v4.0
`lsqvce:gcv:degenerate`	engine.gcvLambda	warning	v4.0
`lsqvce:hatACV:badX`	kernel.hatACV	error	v6.7.0
`lsqvce:jointArcParts:sizeMismatch`	engine.private.jointArcParts	error	v4.0
`lsqvce:jointModelOpt:badG`	engine.jointModelOpt	error	v6.7.0
`lsqvce:jointModelOpt:badMode`	engine.jointModelOpt	error	v4.0
`lsqvce:jointModelOpt:badStream`	engine.jointModelOpt	error	v4.0
`lsqvce:levinson:notPD`	kernel.levinsonDurbin	mixed	v4.0
`lsqvce:levinson:orderTooHigh`	kernel.levinsonDurbin	error	v4.0
`lsqvce:localDiagnostics:noFitted`	engine.localDiagnostics	error	v4.0
`lsqvce:log:noParallelQueue`	util.log	warning	v4.0
`lsqvce:makeKeyFcn:badGroupBy`	engine.private.makeKeyFcn	error	v4.0
`lsqvce:makeLoader:arcCount`	engine.makeLoader	error	v6.0
`lsqvce:makeLoader:badArcs`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:badArcSpec`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:axisConflict`	engine.makeLoader	error	v5.3
`lsqvce:makeLoader:badDay`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:badDayOrder`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:badPrior`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:badScope`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:badValidity`	engine.private.validityOf	error	v4.0
`lsqvce:makeLoader:cepTwice`	engine.makeLoader	error	v5.3
`lsqvce:makeLoader:buildDay`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:dayShape`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:emptyDay`	engine.makeLoader	error	v6.0
`lsqvce:makeLoader:localPlan`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:ncConflict`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:nlConflict`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:noCep`	engine.makeLoader	warning	v5.3
`lsqvce:makeLoader:noArcs`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:noCov`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:noObsTypes`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:paramType`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:prepArrDay`	engine.makeLoader	error	v6.25.0
`lsqvce:makeLoader:prepArrUnused`	engine.makeLoader	error	v6.25.0
`lsqvce:makeLoader:prepArrField`	engine.makeLoader	error	v6.25.0
`lsqvce:makeLoader:priorType`	engine.makeLoader	error	v4.0
`lsqvce:makeLoader:rowWeightSize`	engine.makeLoader	error	v4.0
`lsqvce:mapreduce:badTask`	engine.mapreduce	error	v4.0
`lsqvce:mapreduce:serialFallback`	engine.mapreduce	warning	v4.0
`lsqvce:normalEq:badMerge`	engine.NormalEqSystem	error	v4.0
`lsqvce:normalEq:badWidth`	engine.NormalEqSystem	error	v4.0
`lsqvce:normalEq:cancelation`	engine.NormalEqSystem	warning	v4.0
`lsqvce:normalEq:noData`	engine.NormalEqSystem	error	v4.0
`lsqvce:normalEq:noRedundancy`	engine.NormalEqSystem	error	v4.0
`lsqvce:normalEq:notSPD`	engine.NormalEqSystem	error	v4.0
`lsqvce:normalEq:sizeMismatch`	engine.NormalEqSystem	error	v4.0
`lsqvce:prior:badGroup`	engine.CommonPrior	error	v6.11.0
`lsqvce:prior:badTarget`	engine.CommonPrior	error	v4.0
`lsqvce:probeMatrix:badDist`	engine.private.probeMatrix	error	v4.0
`lsqvce:psd2acv:aboveNyquist`	kernel.psd2acv	warning	v4.0

identifier	raised by	kind	introduced
`lsqvce:psd2acv:nonPositiveVariance`	kernel.psd2acv	error	v4.0
`lsqvce:psd2acv:noZeroFrequency`	kernel.psd2acv	error	v6.31.0
`lsqvce:psd2acv:tooFewPoints`	kernel.psd2acv	error	v4.0
`lsqvce:psdVCE:arcMismatch`	engine.psdVCE	error	v4.0
`lsqvce:psdVCE:shapeMismatch`	engine.psdVCE	error	v4.0
`lsqvce:qr:badMerge`	engine.QRSystem	error	v4.0
`lsqvce:qr:badWidth`	engine.QRSystem	error	v4.0
`lsqvce:qr:cancellation`	engine.QRSystem	warning	v4.0
`lsqvce:qr:noData`	engine.QRSystem	error	v4.0
`lsqvce:qr:noRedundancy`	engine.QRSystem	error	v4.0
`lsqvce:qr:rankDeficient`	engine.QRSystem	error	v4.0
`lsqvce:qr:singularR`	engine.QRSystem	error	v4.0
`lsqvce:qr:sizeMismatch`	engine.QRSystem	error	v4.0
`lsqvce:reduced:notSquare`	engine.ReducedContribution	error	v4.0
`lsqvce:reduced:sizeMismatch`	engine.ReducedContribution	error	v4.0
`lsqvce:redundancy:hutchinsonFailed`	engine.redundancy	warning	v4.0
`lsqvce:redundancy:hutchinsonExplodes`	engine.redundancy	warning	v4.0
`lsqvce:redundancy:identityViolated`	engine.redundancy	warning	v4.0
`lsqvce:redundancy:noStore`	engine.redundancy	error	v4.0
`lsqvce:redundancy:smallGroup`	engine.redundancy	warning	v4.0
`lsqvce:redundancy:spmdNotCollected`	engine.redundancy	warning	v4.0
`lsqvce:redundancyLags:badEll`	kernel.redundancyLags	error	v4.0
`lsqvce:redundancyLags:badLevel`	kernel.redundancyLags	error	v4.0
`lsqvce:redundancyLags:badZ`	kernel.redundancyLags	error	v4.0
`lsqvce:redundancyLags:badW`	kernel.redundancyLags	error	v4.0
`lsqvce:residuals:badSol`	engine.residuals	error	v4.0
`lsqvce:residuals:noStore`	engine.residuals	error	v4.0
`lsqvce:residuals:sigmaMismatch`	engine.residuals	warning	v4.0
`lsqvce:residuals:supersededSol`	engine.residuals	warning	v4.0
`lsqvce:resolveNc:conflict`	engine.resolveNc	error	v4.0
`lsqvce:resolveNc:empty`	engine.resolveNc	error	v4.0
`lsqvce:resolveNc:missing`	engine.resolveNc	error	v4.0
`lsqvce:robust:atBreakdown`	engine.private.warnAtBreakdown	warning	v6.1.4
`lsqvce:robustWeights:badInfluence`	kernel.robustWeights	error	v4.0
`lsqvce:robustWeights:badTuning`	kernel.robustWeights	error	v4.0
`lsqvce:robustWeights:skewFallback`	kernel.robustWeights	warning	v6.1.4
`lsqvce:runMeta:badArg`	engine.runMeta	error	v4.0
`lsqvce:separable:acvTooShort`	kernel.separableWhiten	error	v4.0
`lsqvce:separable:badSigma`	kernel.separableWhiten	error	v4.0
`lsqvce:separable:badSigmaShape`	kernel.separableWhiten	error	v4.0
`lsqvce:separable:epochNotPD`	kernel.separableWhiten	error	v4.0
`lsqvce:separable:notWhitened`	kernel.separableWhiten	error	v4.0
`lsqvce:separable:sigmaLength`	kernel.separableWhiten	error	v4.0
`lsqvce:separable:startNotPD`	kernel.separableWhiten	warning	v4.0
`lsqvce:sharedProbe:badProbes`	engine.sharedProbe	error	v4.0
`lsqvce:sharedProbe:notSquare`	engine.sharedProbe	error	v4.0
`lsqvce:snoop:breakdown`	engine.verdict	error	v4.0
`lsqvce:snoop:contaminated`	engine.verdict	warning	v4.0
`lsqvce:snoop:correlatedBatch`	engine.verdict	error	v4.0
`lsqvce:snoop:downdateFailed`	engine.verdict	warning	v4.0
`lsqvce:snoop:forcedBreakdown`	engine.verdict	warning	v4.0
`lsqvce:snoop:forcedContaminated`	engine.verdict	warning	v4.0
`lsqvce:snoop:forcedOverlap`	engine.verdict	warning	v4.0
`lsqvce:snoop:indeterminate`	engine.verdict	warning	v6.23.0

identifier	raised by	kind	introduced
`lsqvce:snoop:lowRedundancy`	engine.verdict	warning	v4.0
`lsqvce:snoop:maxRejectFraction`	engine.verdict	warning	v4.0
`lsqvce:snoop:rejectFraction`	engine.verdict	warning	v4.0
`lsqvce:snoop:tooContaminated`	engine.verdict	error	v4.0
`lsqvce:solveContributions:badContributions`	solveGdContributions	error	v6.35.0
`lsqvce:solveContributions:badMultiplicity`	solveGdMultiplicity	error	v6.35.0
`lsqvce:stat:badDof`	kernel.stat	error	v4.0
`lsqvce:stat:badProb`	kernel.stat	error	v4.0
`lsqvce:stat:sizeMismatch`	kernel.stat	error	v4.0
`lsqvce:streamContribution:badScale`	engine.StreamContribution	error	v4.0
`lsqvce:streamContribution:notPD`	engine.StreamContribution	error	v4.0
`lsqvce:streamContribution:noAccum`	engine.StreamContribution	error	v4.0
`lsqvce:streamContribution:noStore`	engine.StreamContribution	error	v4.0
`lsqvce:synthLoader:badArg`	engine.synthLoader	error	v4.0
`lsqvce:synthLoader:noNoiseModel`	engine.synthLoader	error	v4.0
`lsqvce:synthLoader:robustFactor`	engine.synthLoader	error	v6.22.0
`lsqvce:systematic:badCoverage`	engine.verdict	error	v4.0
`lsqvce:systematic:lbUnreliable`	engine.verdict	warning	v4.0
`lsqvce:systematic:noResiduals`	engine.verdict	error	v4.0
`lsqvce:systematic:tooFew`	engine.verdict	warning	v4.0
`lsqvce:toeplitzCofactor:badVariance`	kernel.ToeplitzCofactor	warning	v4.0
`lsqvce:toeplitzCofactor:empty`	kernel.ToeplitzCofactor	error	v4.0
`lsqvce:toeplitzCofactor:notACV`	kernel.ToeplitzCofactor	warning	v4.0
`lsqvce:traceEstimate:badOperator`	engine.traceEstimate	error	v4.0
`lsqvce:traceEstimate:negSize`	engine.traceEstimate	error	v4.0
`lsqvce:traceEstimate:nonSquare`	engine.traceEstimate	error	v4.0
`lsqvce:traceEstimate:sizeMismatch`	engine.traceEstimate	error	v4.0
`lsqvce:truncationDeficit:badSize`	kernel.truncationDeficit	error	v6.34.0
`lsqvce:util:log:badKey`	util.log	error	v4.0
`lsqvce:varACV:badA`	kernel.varACV	error	v4.0
`lsqvce:varACV:notStationary`	kernel.varACV	error	v4.0
`lsqvce:varWhiten:badA`	kernel.varWhiten	error	v4.0
`lsqvce:varWhiten:badSigma`	kernel.varWhiten	error	v4.0
`lsqvce:varWhiten:badStartup`	kernel.varWhiten	error	v4.0
`lsqvce:varWhiten:notWhitened`	kernel.varWhiten	error	v4.0
`lsqvce:vce:badAccFcn`	engine.vce	error	v4.0
`lsqvce:vce:badSigma0`	engine.vce	error	v4.0
`lsqvce:vce:groupMismatch`	engine.vce	error	v4.0
`lsqvce:vce:hutchGroupMismatch`	engine.vce	error	v4.0
`lsqvce:vce:lowRedundancy`	engine.vce	warning	v4.0
`lsqvce:vce:notConverged`	engine.vce	warning	v4.0
`lsqvce:verbosity:badLevel`	util.verbosity	error	v4.0
`lsqvce:verbosity:badTag`	util.verbosity	error	v4.0
`lsqvce:wSnoop:badSize`	kernel.wSnoop	error	v6.34.0
`lsqvce:whittle:badG`	kernel.whittle	error	v4.0
`lsqvce:whittle:notPD`	kernel.whittle	error	v4.0
`lsqvce:covModel:badNugget`	engine.covModel	error	v6.6.0
`lsqvce:covModel:nuggetNeedsWhiteMod`	engine.covModel	error	v6.6.0
`lsqvce:redundancyLags:badNegLag`	kernel.redundancyLags	error	v6.6.0
`lsqvce:accum:noRedundancy`	engine.BlockAccumulator	error	v6.20.0
`lsqvce:accum:notBlockDiagonal`	engine.BlockAccumulator	warning	v6.20.0
`lsqvce:accum:duplicateDay`	engine.BlockAccumulator	error	v6.22.0
`lsqvce:accum:mixedIdTypes`	engine.BlockAccumulator	error	v6.22.0
`lsqvce:prior:zeroRow`	engine.CommonPrior	error	v6.20.0

identifier	raised by	kind	introduced
`lsqvce:arcVarianceFactors:nonPositive`	engine.arcVarianceFactors	warning	v6.21.0
`lsqvce:armaACV:truncated`	kernel.armaACV	warning	v6.21.0
`lsqvce:covModel:unevenGrid`	engine.covModel	warning	v6.21.0
`lsqvce:crossCorr:badLength`	kernel.crossCorr	error	v6.21.0
`lsqvce:gcv:badData`	engine.gcvLambda	error	v6.24.0
`lsqvce:gcv:nullUnobserved`	engine.gcvLambda	error	v6.24.0
`lsqvce:TaskDefinition:SystemDefinition`	SystemDefinition, TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:SysName`	SysNameDefinition	error	v7.0.0
`lsqvce:TaskDefinition:TaskDefId`	TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:TaskRefs`	TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:TaskDefId`	TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:SystemDefinition`	SystemDefinition, TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:TaskDefId`	TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:TaskDefId`	TaskDefinition	error	v7.0.0
`lsqvce:TaskDefinition:SysArcs`	SysNameDefinition	error	v7.0.0
`lsqvce:TaskDefinition:SystemDefs`	SystemDefinition, TaskDefinition	error	v7.0.0
`lsqvce:DayContext:unknownName`	DayContext	error	v7.0.0
`lsqvce:DayContext:arcRequired`	DayContext	error	v7.0.0
`lsqvce:DayContext:mixedDayCons`	DayContext	error	v7.0.0
`lsqvce:DayContext:badRefDay`	DayContext	error	v7.0.0
`lsqvce:DayContext:localDayDB`	DayData	error	v7.0.0
`lsqvce:DayContext:badLocDay`	DayData	error	v7.0.0
`lsqvce:DayContext:badADay`	DayData	error	v7.0.0